UVSQ & LEDGER DONJON

Master thesis

SNARKs in Constrained Systems: A Study on Verifier Implementation

---

Mareau Quentin

Supervised by Yannick Seurin

Year 2026

Abstract

This report was written during a cryptography internship at Ledger, whose main goal was to study how realistic it would be to implement a Verifier on one of their devices. The work first surveys SNARKs and existing implementations, what they offer, and what obstacles arise for the implementation under consideration. This led me to study the memory required by such a Verifier program and how to manage it, which are the main subjects of this report. In parallel, I wrote a side report [Mar26] about the construction of a SNARK based on the STARK approach [BSBHR18b], which helped me understand the parameters involved in the implementations I studied.

CONTENTS

1 INTRODUCTION

SNARKs, short for *Succinct Non-interactive ARgument of Knowledge*, formalized by [BCCT11], address the problem of delegated computation. More specifically, a classical model considers a local device, often less powerful, that asks a remote device, often more powerful, to execute a program and send back the result. In this context, the first device would like to be convinced that the result it receives is indeed the result of the agreed-upon program, executed on the agreed-upon inputs. The goal is to prevent two kinds of failures:

- unexpected computation errors by the remote device;
- malicious misreporting by the remote device.

Without any particular efficiency constraint, the simplest solution is for the local device to recompute the result itself. SNARKs address additional constraints through the following model:

1. The remote device, called the Prover, performs the intended computation if it is honest and generates a proof for this computation. This proof must be small compared with the computation performed and is transmitted in addition to the result. Note that this proof depends on the program executed by the Prover, on the inputs, and therefore indirectly on the output.
2. The local device, called the Verifier, receives the result and the proof, then checks the validity of this pair at low cost.

A *zero-knowledge* aspect is often added to these constraints, so that the proof reveals no unnecessary information about the program's *private* inputs. A classical example is based on the following assertion: "For a fixed hash function and a public hash h, the Prover knows an x such that $H(x) = h$." The Prover's goal is then to run an implementation of the hash function and send a proof that a correct execution returned h, without providing any other information about x. This property is often highlighted in concrete application projects such as confidential identity proofs, private transactions, or cloud *verifiable computing*.

This field has evolved substantially over the last ten years, notably with the arrival of proof systems such as Groth16 [Gro16] and PLONK [GWC19]. These proof systems offer fast proof generation and verification, as well as constant and very small proof sizes; they therefore provide a strong answer to the problems stated above. However, they have two major drawbacks:

- they rely on elliptic-curve assumptions, which makes them vulnerable to quantum attacks;
- they depend on a *trusted setup*, meaning that the system must first be initialized with common parameters. If any secret randomness generated during this setup, often called *toxic waste*, were retained, the security of the protocol could be compromised.

This motivated the introduction of **STARKs**, for *Scalable Transparent ARgument of Knowledge*, in [BSBHR18b]. In their non-interactive version, they can be seen as transparent and scalable SNARKs, meaning in particular that they offer quasi-linear proving time for the Prover. These proof systems do not require a *trusted setup* and often rely on assumptions believed to be compatible with post-quantum security. However, these advantages come with a major drawback: the proof size is no longer constant, and proofs are much larger than for PLONK and Groth16.

A SNARK-based approach can be highly relevant to a company such as Ledger, which motivates this study of the feasibility of running the Verifier on its devices. In this model, some non-sensitive computations can be delegated off-device, while their results are verified on a trusted device. Consequently, even when a result informs a security-sensitive decision by the user, the user can gain confidence that the

data displayed on the device correspond to a successfully verified computation. Although this model is attractive, it is subject to significant hardware constraints. Indeed, Ledger devices all use the architecture described in Figure 1, where most sensitive computations are performed in the Secure Element. This hardware is subject to important constraints, notably memory capacity and computational performance: for instance, the Ledger Flex device uses a ST33K1M5C Secure Element, which is clocked at 70 MHz and includes 64 kB of user RAM.

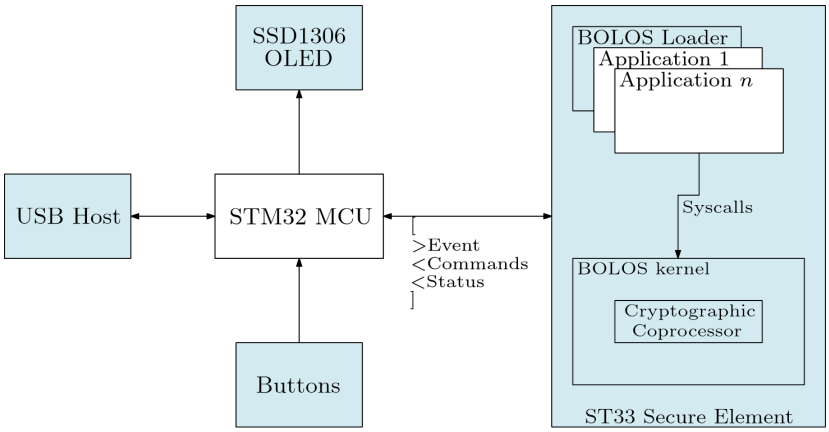


Figure 1: Ledger OS architecture

Although many benchmarks exist for proof systems, most of them focus on Prover resources, verification time, and proof size. While these metrics, particularly proof size, are relevant, they are insufficient for this study. We therefore consider the RAM consumption of the Verifier program, but also a metric that is often omitted: the size of the *Verifier artifacts*. These artifacts are the circuit-specific data that must be available on the device during verification to perform certain computations.

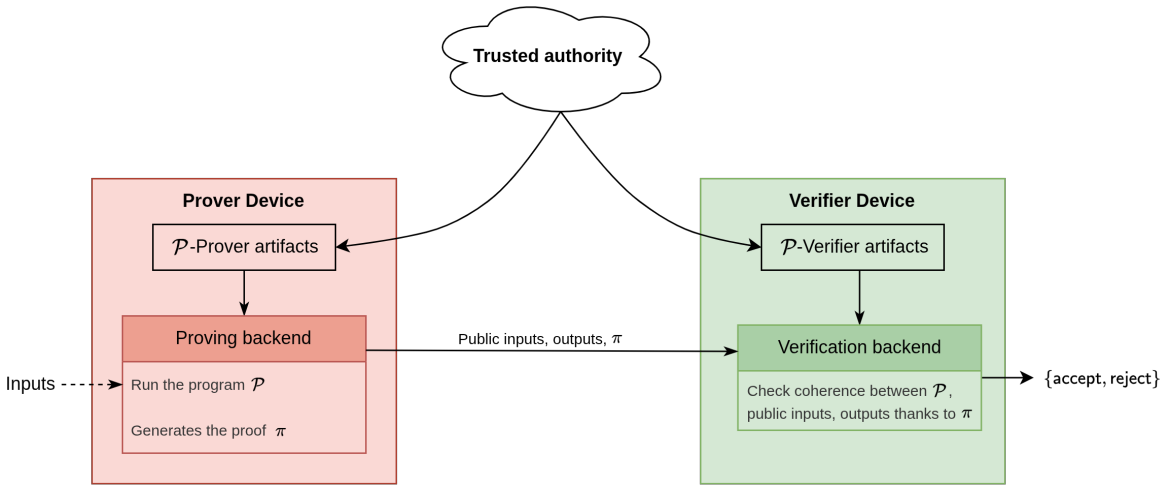


Figure 2: Representation of the Prover/Verifier model

These artifacts are often overlooked in existing analyses because, although they can reach several dozen megabytes in some systems, this is usually manageable on a recent computer. In our model, however, these sizes are incompatible with the memory capacity of a Secure Element.

The proof systems considered in this work are: Barretenberg, Binius64, OpenVM, Circom, Flock, Plonky2, RISC Zero, SP1, Spartan2. The choice of these implementations is motivated by the variety of arithmetization techniques they use and by their potential applications, but also by the aim of contributing to the useful benchmark work provided by ethproofs (see their benchmark

repository). The following compact classification of these proof systems will be used later to analyze Verifier costs:

Tool name	Arithmetization	SNARK protocol	Post-Quantum	VM	Security[1]
Barretenberg	Plonkish	UltraHonk/KZG	✗	✗	100 bits
Binius64	Binius64	Iron Spartan/BaseFold	✓	✗	96 bits
Circom	R1CS	Groth16	✗	✗	100 bits
Flock	R1CS	Flock/BaseFold	✓	✗	100 bits
OpenVM†	AIR	SWIRL/WHIR	✓	✓	100 bits
Plonky2	Plonkish	FRI	✓	✗	97 bits
RISC Zero‡	AIR	STARK	✓	✓	96 bits
SP1‡	AIR	Hypercube/Jagged	✓	✓	100 bits
Spartan2	R1CS	Spartan/Hyrax	✗	✗	128 bits

Table 1: Proof-system properties

Here are a few details about selected proof systems used in this work:

† We used the STARK backend directly with SHA-256-specific AIR objects instead of executing the program in full OpenVM zkVM. This makes OpenVM closer to a "direct AIR" benchmark.

‡ Both proof systems were measured in their recursive compressed modes. The external verifier artifact is therefore tiny, but universal AIR / recursion verifier material is embedded in the verifier implementation, and proof sizes are mode-specific.

Our results[2] show that, once we consider post-quantum and transparent proof systems, there is a trade-off between proof size, Verifier artifact size, and proving time. We will also present some solutions to the problems encountered, even if they cannot eliminate this kind of compromise. Ultimately, this study led to the choice of a proof system for a proof-of-concept application running on a Ledger Flex: Plonky2-Display . This application embeds a Verifier program that runs directly on the Flex's Secure Element and can verify computations for two programs, including SHA-256.

2 Preliminaries

We first present some elementary tools and procedures used for the construction of various SNARKs. For now, these definitions are abstract in order to remain as general as possible. Nevertheless, it is useful to keep in mind that there are almost as many tools as there are proof systems. Thus, the choice of any of them is motivated by specific optimizations related to the field choice, the program being proven, scalability, or other properties.

2.1 Relations and Interactive Oracle Proofs

As presented in the introduction, the arithmetization step, which will be detailed later, transforms the correct computation of a program into the knowledge of mathematical objects satisfying a *relation*.

[1] These security levels are the values advertised by the respective projects and some of them might be based on outdated results.

[2] All measurements were performed on SHA-256 programs with 128- and 256-byte inputs, using an Intel Core i7-1165G7 CPU and 32 GB of RAM

Definition 2.1. A **relation** is a subset $\mathcal{R} \subseteq \{0,1\}^* \times \{0,1\}^*$. If $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, we say that x is an *instance* and that w is a *witness* for x. For $\mathsf{x} \in \{0,1\}^*$, we denote $\mathcal{R}|_{\mathsf{x}} = \{\mathsf{w} \in \{0,1\}^* \mid (\mathsf{x},\mathsf{w}) \in \mathcal{R}\}$.

Definition 2.2. The **language** associated with a relation $\mathcal{R}$ is

$$\mathcal{L}(\mathcal{R}) = \{\mathsf{x} \in \{0,1\}^* \mid \exists \mathsf{w} \in \{0,1\}^*,\ (\mathsf{x},\mathsf{w}) \in \mathcal{R}\}.$$

Definition 2.3. An interactive protocol is said to be *public-coin* if all messages sent by the Verifier consist of uniformly random challenges sampled from public randomness.

Definition 2.4. An *interactive oracle proof*, abbreviated as **IOP**, for a relation $\mathcal{R}$, with soundness error $\varepsilon : \{0,1\}^* \to [0,1]$ and number of rounds $k : \{0,1\}^* \to \mathbb{N}$, is a pair of probabilistic polynomial-time interactive algorithms (P,V) such that:

0. **Setup:** for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover receives (x,w) and the Verifier receives x;
1. **Interaction:** at each round $i \in \{1,\dots,k(\mathsf{x})\}$, the Verifier sends a message α_i and the Prover sends an *oracle message* m_i, which the Verifier does not read in full;
2. **Queries:** the Verifier queries the oracle messages sent by the Prover at some locations. At the end of the protocol, the Verifier returns accept or reject;
3. **Completeness:** for every $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$,
$$\mathbb{P}\left[\langle P(\mathsf{x},\mathsf{w}),\ V(\mathsf{x})\rangle = \mathsf{accept}\right] = 1;$$
4. **Soundness:** for every $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ and every possibly unbounded malicious Prover $\tilde{P}$, we have
$$\mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\ V(\mathsf{x})\right\rangle = \mathsf{accept}\right] \leq \varepsilon(\mathsf{x}).$$

We denote by $p(\mathsf{x}) = \sum_{i=1}^{k(\mathsf{x})} |m_i|$ the *proof size* and by $q(\mathsf{x})$ the total number of entries read by V, called the *query complexity*.

Remark. An IOP can be viewed as a compromise between an interactive proof and a PCP (see [CY24]): interaction is preserved, but the Prover's long messages are treated as oracles that the Verifier probes at only a small number of positions. These queries are generally adaptive and may depend on the entire preceding transcript. In what follows, we are interested exclusively in public-coin IOPs, a condition which is necessary for compilation by BCS/Fiat-Shamir into a non-interactive argument.

Definition 2.5 (Argument of knowledge)**.** An IOP (P,V) for a relation $\mathcal{R}$ is an *argument of knowledge* with error (or *knowledge-soundness error*) $\mathsf{e} : \{0,1\}^* \longrightarrow [0,1]$ if there exists a probabilistic polynomial-time extractor E such that, for every instance x and every malicious Prover $\tilde{P}$, we have

$$\mathbb{P}\left[(\mathsf{x}, E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}\right] \ \geq\ \mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\ V(\mathsf{x})\right\rangle = \mathsf{accept}\right] - \mathsf{e}(\mathsf{x}),$$

where $E^{\tilde{P}}$ denotes E with access to the transcript of $\tilde{P}$ at every stage of the protocol (but not to its auxiliary inputs or to its internal randomness).

Remark. For $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$, we observe that if our IOP is an argument of knowledge, then

$$\mathbb{P}\left[\left\langle \tilde{P}(\mathsf{x}),\ V(\mathsf{x})\right\rangle = \mathsf{accept}\right] \leq \mathsf{e}(\mathsf{x}).$$

Thus, this property strengthens the usual soundness: a Prover who manages to convince the Verifier must in fact *know* a witness, in the sense that a witness can be extracted from him. This notion is essential for SNARKs, which are by definition *arguments of knowledge*.

2.2 SNARKs and the BCS transformation

The acronym **SNARK** highlights complexity properties of a non-interactive *argument of knowledge*. We adopt here the following convention, compatible with the standard terminology of succinct arguments [Gro16] and with the literature derived from IOPs [BSCS16, BSBHR18b]. The properties of completeness, soundness, and extractability are the non-interactive analogues of those introduced for IOPs. Throughout the rest of this report, λ denotes the security parameter, x the public instance, w a witness for x, and $T := T(\mathsf{x},\mathsf{w})$ denotes a measure of the size of the proven computation.

Definition 2.6 (SNARK)**.** Let $\mathcal{R}$ be a relation. A **SNARK** (*Succinct Non-interactive ARgument of Knowledge*) for $\mathcal{R}$ is a triple of algorithms:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$: takes as input the security parameter λ and returns public parameters pp;
- $\pi \leftarrow \mathsf{Prove}(\mathsf{pp},\mathsf{x},\mathsf{w})$: takes as input pp, an instance $\mathsf{x} \in \mathcal{L}(\mathcal{R})$ and $\mathsf{w} \in \mathcal{R}|_{\mathsf{x}}$, and returns a *proof* $\pi \in \{0,1\}^*$;
- $b \leftarrow \mathsf{Verify}(\mathsf{pp},\mathsf{x},\pi)$: takes as input pp, an instance x and a word π, and returns accept or reject.

These algorithms must satisfy completeness, knowledge soundness (which implies computational soundness), and **succinctness**: for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$ of computation size T,

$$|\pi| \leq \mathsf{poly}(\lambda, \log T) \qquad \text{and} \qquad \mathrm{Time}\big(\mathsf{Verify}(\mathsf{pp},\mathsf{x},\pi)\big) = O(|\mathsf{x}|, \mathsf{poly}(\lambda, \log T)),$$

where $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda)$.

Provided that proof sizes and verification times are sufficiently bounded, the notion of IOP with argument of knowledge studied above seems particularly well suited to the construction of SNARKs. However, it is first necessary to make this kind of protocol *non-interactive*:

Definition 2.7 (Non-interactive Oracle Proof)**.** A *non-interactive oracle proof* for a relation $\mathcal{R}$, with soundness error $\varepsilon : \{0,1\}^* \times \mathbb{N}^* \to [0,1]$, is a pair of probabilistic polynomial-time algorithms (P,V) such that:

1. for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover receives (x,w) and the Verifier receives x;
2. **Non-interactivity:** for every honest input $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$, the Prover simulates an IOP with Q queries using a random oracle H and produces a single message π, which he sends to the Verifier;
3. the algorithm $V^H(\mathsf{x},\pi)$ returns either accept or reject;
4. **Completeness:** for every $(\mathsf{x},\mathsf{w}) \in \mathcal{R}$,
$$\mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow P^H(\mathsf{x},\mathsf{w})\right] = 1;$$
5. **Soundness:** for every $\mathsf{x} \notin \mathcal{L}(\mathcal{R})$ and every malicious Prover $\tilde{P}$ making Q queries, possibly unbounded, we have
$$\mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow \tilde{P}^H(\mathsf{x})\right] \leq \varepsilon(\mathsf{x},Q).$$

We will say that this protocol is an argument of knowledge with error $\mathsf{e} : \{0,1\}^* \times \mathbb{N}^* \longrightarrow [0,1]$ if there exists a probabilistic polynomial-time extractor E such that, for every instance x and every malicious Prover $\tilde{P}$ making Q queries, we have

$$\mathbb{P}\left[(\mathsf{x}, E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}\right] \geq \mathbb{P}\left[V^H(\mathsf{x},\pi) = \mathsf{accept} \mid H \leftarrow \mathcal{U}(\lambda),\ \pi \leftarrow \tilde{P}^H(\mathsf{x})\right] - \mathsf{e}(\mathsf{x},Q).$$

Remark. Above, the notation $\mathcal{U}(\lambda)$ denotes the uniform distribution over the set of functions $H : \{0,1\}^* \longrightarrow \{0,1\}^\lambda$ and therefore $H \leftarrow \mathcal{U}(\lambda)$ denotes a random oracle.

The naive approach is the direct use of the Fiat-Shamir procedure from classical interactive protocols. In this setting, one fixes a random oracle H and each step proceeds as follows:

- the Prover has constructed his message to the Verifier m_{i-1} at the previous step;
- he simulates the random challenge $\alpha_i = H(\mathsf{x} \,\|\, \alpha_{i-1} \,\|\, m_{i-1})$, or $\alpha_1 = H(\mathsf{x})$ if $i = 1$;
- he constructs m_i as he would have done during the interactive protocol.

The final proof provided by the Prover to the Verifier is then $\pi = (m_1, \ldots, m_k)$. The Verifier can then reconstruct the α_i and perform his usual verification.

This approach does not adapt directly to IOPs, because in this model the Verifier is not supposed to read the messages m_i in full, but only to query them at a small number of positions. For this reason, [BSCS16] introduced a refinement of Fiat-Shamir adapted to IOPs, later called the *BCS transformation*. The method uses Merkle commitments (see [CY24]) to make the oracle messages binding while still allowing the Verifier to inspect only a few entries. As in the Fiat-Shamir transformation, the Verifier's random challenges are generated from a random oracle applied to the transcript. In addition, the same transcript determines the query positions to be opened. The resulting proof consists of Merkle roots, the values opened at the queried positions, the corresponding authentication paths, and the random-oracle transcript from which the challenges and query positions are recomputed.

We now present the main result, from [BSCS16, Theorem 7.1] and [BGK+23, Theorem 3.15], which justifies the construction of SNARKs from IOPs. We refer to these articles for a more formal and precise description of the technique.

Theorem 2.1 (Properties of BCS)**.** *Let $\mathcal{R}$ be a relation and let (P,V) be a public-coin IOP with $k := k(\mathsf{x})$ rounds, $q := q(\mathsf{x})$ queries, round-by-round soundness $(\varepsilon_1, \ldots, \varepsilon_k)$, and proof size $p := p(\mathsf{x})$. We also denote by T_P and T_V the respective time complexities of the Prover and the Verifier, and by Q the maximum number of queries simulated by an adversary. Then the non-interactive protocol obtained from the BCS transformation, defined for a security parameter λ, satisfies:*

(i) ***Soundness.*** $\varepsilon'(\mathsf{x}, Q, \lambda) = Q \cdot \max_i \varepsilon_i + 3(Q^2+1) \cdot 2^{-\lambda}$;

(ii) ***Proof size.*** $p'(\mathsf{x}, \lambda) = (k + q \cdot (\lceil \log_2 p(\mathsf{x}) \rceil + 2) + 1) \cdot \lambda$;

(iii) ***Proving time.*** $T'_P(\mathsf{x}, \lambda) = \mathsf{poly}(\lambda) \cdot O(k+p) + T_P + T_V$;

(iv) ***Verification time.*** $T'_V(\mathsf{x}, \lambda) = \mathsf{poly}(\lambda) \cdot O(k+q) + T_V$.

Moreover, if (P,V) is an argument of knowledge with error e, then so is its non-interactive version,

$$\mathsf{e}'(\mathsf{x}, Q, \lambda) = Q \cdot \mathsf{e}(\mathsf{x}) + 3(Q^2+1) \cdot 2^{-\lambda}.$$

As announced, this result allows us to restrict our construction to that of an IOP with argument of knowledge satisfying the complexity constraints of SNARKs. In that case, one must ensure that the relation associated with our IOP establishes a link with the correct execution of a program. This will be studied in Section 3.

2.3 Polynomial Commitment Schemes

A *Polynomial Commitment Scheme* (PCS) allows a Prover to commit to a polynomial of bounded degree and then convince a Verifier that a public tuple of values indeed corresponds to the evaluations of the committed polynomial at a public tuple of points, without revealing the entire polynomial. This cryptographic primitive first appeared in [KZG10] in 2010 and has since been refined into various adaptations, each suited to its own context. It is a fundamental procedure used in most IOP and SNARK constructions. In the setting of interest to us, we adopt a *batched* definition inspired by that of [Fu25], in which the evaluation algorithm is a public-coin interactive protocol allowing several evaluations to be queried simultaneously.

Definition 2.8. Let $\mathbb{F}$ be a field and let $d, m_{\max} \in \mathbb{N}$, where d is an upper bound on the degree of the accepted polynomials and $m_{\max}$ is an upper bound on the number of simultaneous evaluation queries. A *polynomial commitment scheme* over $\mathbb{F}$ is a quadruple of probabilistic polynomial-time algorithms

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max})$: takes as input the security parameter λ, the bound d, and the bound $m_{\max}$, and returns public parameters pp;
- $C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X))$: takes as input pp, d, and a polynomial $f \in \mathbb{F}^{<d}[X]$, and returns a commitment C;
- $b \in \{0,1\} \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f(X))$: takes as input pp, d, C, and $f(X)$, and returns 1 if $f(X)$ is accepted by the opening relation associated with C, and 0 otherwise;
- $b \in \{0,1\} \leftarrow \mathsf{Eval}(\mathsf{pp}, d, C, \Omega, \mathbf{y}; f(X))$: denotes a public-coin interactive protocol

$$\langle P(\mathsf{pp}, d, C, \Omega, \mathbf{y}, f(X)),\ V(\mathsf{pp}, d, C, \Omega, \mathbf{y}) \rangle .$$

Here we denote

$$\Omega = (z_1, \ldots, z_m) \in \mathbb{F}^m \qquad \text{with} \qquad m \le m_{\max},$$

where the z_i are pairwise distinct, and

$$\mathbf{y} = (y_1, \ldots, y_m) \in \mathbb{F}^m.$$

The Prover and the Verifier have pp, d, C, Ω, and $\mathbf{y}$ as common inputs. The Prover additionally knows a polynomial $f(X)$ such that $\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1$. At the end of the protocol, the Verifier returns a bit b such that $b = 1$ if the claim

$$f(z_i) = y_i \qquad \text{for every } i \in \{1, \ldots, m\}$$

is accepted, and $b = 0$ otherwise.

This quadruple must satisfy the following properties:

1. **Completeness.** For all $\lambda, d, m_{\max} \in \mathbb{N}$, every polynomial $f \in \mathbb{F}^{<d}[X]$, and every admissible tuple

$$\Omega = (z_1, \ldots, z_m) \in \mathbb{F}^m$$

of pairwise distinct points, with $m \le m_{\max}$, we have

$$\mathbb{P}\left[\langle P(\mathsf{pp}, d, C, \Omega, \mathbf{y}, f(X)),\ V(\mathsf{pp}, d, C, \Omega, \mathbf{y}) \rangle = 1 \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max}) \\ C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X)) \\ \mathbf{y} = (f(z_1), \ldots, f(z_m)) \end{array} \right] = 1.$$

2. **Binding.** For every probabilistic polynomial-time adversary $\mathcal{A}$, we have

$$\mathbb{P}\left[b_1 = b_2 = 1 \wedge f_1(X) \neq f_2(X) \;\middle|\; \begin{array}{c} \mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max}) \\ (C, d, f_1(X), f_2(X)) \leftarrow \mathcal{A}(\mathsf{pp}) \\ b_1 \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) \\ b_2 \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) \end{array} \right] \le \mathrm{negl}(\lambda).$$

3. **Knowledge extractability.** The PCS satisfies knowledge extractability if the protocol Eval is an argument of knowledge for the relation

$$\mathcal{R}_{\mathsf{Eval}} = \left\{ \big((\mathsf{pp}, d, C, \Omega, \mathbf{y}), f(X)\big) \;\middle|\; \begin{smallmatrix} \Omega=(z_1,\ldots,z_m)\in\mathbb{F}^m \text{admissible} \\ \mathsf{Open}(\mathsf{pp},d,C,f(X))=1 \\ f(z_i)=y_i \text{ for every } i\in\{1,\ldots,m\} \end{smallmatrix} \right\}.$$

In other words, every Prover who convinces the Verifier in Eval is able to construct $f \in \mathbb{F}^{<d}[X]$ in polynomial time such that $\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1$ and

$$f(z_i) = y_i \qquad \text{for every } i \in \{1, \ldots, m\}.$$

Remark. The algorithm Open should be understood as the opening relation specified by the considered scheme. In classical schemes, this relation may express that C is an exact commitment to f. In proximity-based schemes, such as the FRI-based construction recalled in Appendix A, it may instead express that the object bound by C is sufficiently close to the evaluation of f over a prescribed domain. The binding property below is what ensures that this opening relation cannot accept two distinct polynomials for the same commitment, except with negligible probability.
In the context of zero-knowledge proofs, one may also require that the protocol satisfy a *hiding* property, meaning that no information about the polynomial is revealed through the interactions.

Proposition 2.2 (Opening consistency)**.** *Consider a PCS over $\mathbb{F}$. For every probabilistic polynomial-time adversary $\mathcal{A}$, we have*

$$\mathbb{P}\left[\begin{matrix} \Big((\mathsf{pp},d,C,\Omega_1,\mathbf{y}_1),f_1(X)\Big)\in\mathcal{R}_{\mathsf{Eval}} \\ \Big((\mathsf{pp},d,C,\Omega_2,\mathbf{y}_2),f_2(X)\Big)\in\mathcal{R}_{\mathsf{Eval}} \\ f_1(X)\neq f_2(X) \end{matrix} \;\middle|\; \begin{matrix} \mathsf{pp}\leftarrow\mathsf{Setup}(1^\lambda,d,m_{\max}) \\ (C,\Omega_1,\mathbf{y}_1,\Omega_2,\mathbf{y}_2,f_1(X),f_2(X))\leftarrow\mathcal{A}(\mathsf{pp}) \end{matrix} \right] \leq \mathrm{negl}(\lambda).$$

Proof. Suppose that the considered event occurs. By definition of the relation $\mathcal{R}_{\mathsf{Eval}}$, we then have

$$\mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) = 1 \qquad \text{and} \qquad \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) = 1,$$

as well as

$$f_1(X) \neq f_2(X).$$

We then construct a probabilistic polynomial-time adversary $\mathcal{A}'$ against the binding property as follows: on input pp, it runs $\mathcal{A}(\mathsf{pp})$, obtains

$$(C, \Omega_1, \mathbf{y}_1, \Omega_2, \mathbf{y}_2, f_1(X), f_2(X)),$$

then returns

$$(C, d, f_1(X), f_2(X)).$$

As soon as the event above occurs for $\mathcal{A}$, the adversary $\mathcal{A}'$ therefore produces two distinct polynomials $f_1(X)$ and $f_2(X)$ such that

$$\mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) = \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) = 1.$$

This contradicts the binding property of the PCS. The probability that the considered event occurs is therefore negligible in λ. □

Overall, we can summarize a SNARK construction by the following diagram:

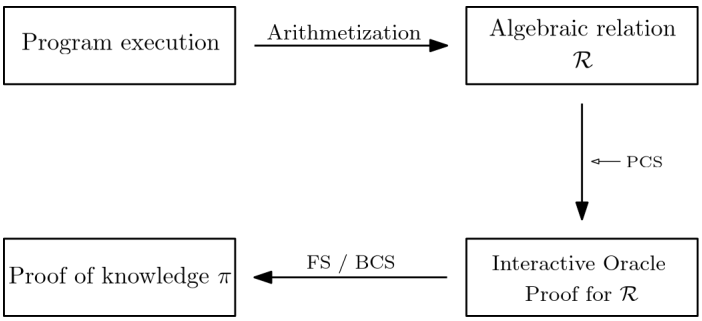


Figure 3: Construction of a SNARK

However, it is important to note that this construction is not universal and remains flexible depending on the arithmetization, the protocols used, and the targeted constraints.

3 ARITHMETIZATION AND VERIFIER ARTIFACTS

The tools and protocols used in the construction of proof systems allow the Prover to convince the Verifier that the Prover knows algebraic objects satisfying certain properties. However, the execution of a program must first be connected to such objects. This is precisely the role of arithmetization, which is the subject of this section. Although the boundary between arithmetization and proving tools may sometimes be blurry, the goal of this section is to introduce procedures that translate a correct execution into algebraic relations, as illustrated in Figure 3. There exist different approaches for this step, each with its own advantages and limitations; therefore, we will present only three common families of arithmetizations.

3.1 R1CS: Rank-1 constraint system

One way to represent a program is with *arithmetic circuits*, which take as inputs either variables or constants over a finite field $\mathbb{F}$ and use only addition and multiplication gates:

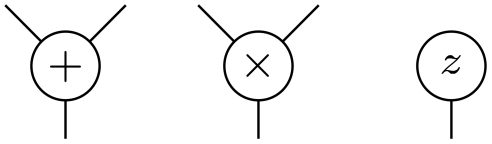


Figure 4: Allowed elementary gates for arithmetic circuits

We can also require a circuit output to equal a constant in order to represent equations. For instance, for $z \in \mathbb{F}$, a standard way to test whether $z \in \{0,1\}$ is to compute $z(z-1)$ and check that it vanishes. Thus, we can build the following is_bool circuit that represents this equation:

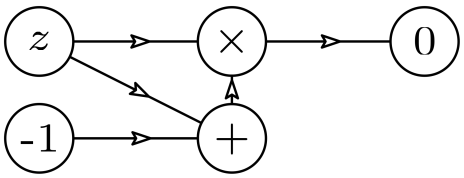
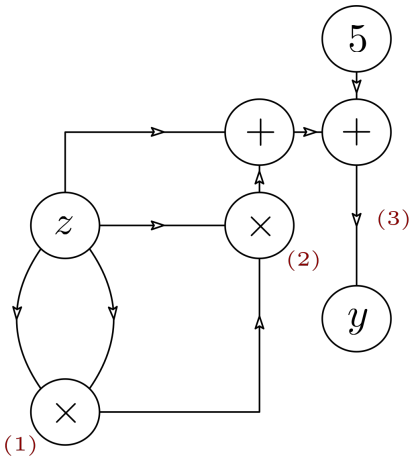


Figure 5: is_bool circuit

Therefore, if the output of this circuit vanishes when evaluated at $z \in \mathbb{F}$, then $z \in \{0,1\}$. More generally, we consider a program f[3] that takes inputs $I_1, \ldots, I_j$ (some of which may be public) and returns public outputs $\mathsf{out}_1, \ldots, \mathsf{out}_k$. The next step of the R1CS arithmetization is to translate that circuit into a system of equations with only one (possibly trivial) product.

Example. Let us consider the program $f(z) = z^3 + z + 5$, with output $\mathsf{out} = y$. The associated circuit and its translation into a system of equations are represented below:

[3] assumed to be represented as an arithmetic circuit



$$\longleftrightarrow \qquad (\mathcal{S}_f) \begin{cases} s_1 = z \times z & (1) \\ s_2 = s_1 \times z & (2) \\ y = (s_2 + z + 5) \times 1 & (3) \end{cases}$$

For $z \in \mathbb{F}$ such that $f(z) = y$, evaluating $f(z)$ is equivalent to finding (z, s_1, s_2) that satisfy $(\mathcal{S}_f)$.

Figure 6: Circuit representation of f

We may collect every wire value, including inputs, outputs, and intermediate values, together with a leading constant 1 in a vector $\mathbf{a} = (\mathbf{x}, \mathbf{w}) \in \mathbb{F}^M$, where $\mathbf{x} \in \mathbb{F}^\ell$ represents the public statement and $\mathbf{w} \in \mathbb{F}^{M-\ell}$ represents a private witness. The next step is then to represent the system $(\mathcal{S}_f)$ using matrices $L, R, O \in \mathcal{M}_{N \times M}(\mathbb{F})$, where N is the number of equations of $(\mathcal{S}_f)$. Since every equation is written with exactly one multiplication symbol, these matrices respectively encode the left operand, the right operand, and the output of each equation.

Example. In our example, we have $\mathbf{x} = (1, y)$ and $\mathbf{w} = (z, s_1, s_2)$. We can then describe the matrix L as:

$$\begin{cases} z \times z & = s_1 \\ s_1 \times z & = s_2 \\ (s_2 + z + 5) \times 1 & = y \end{cases} \longrightarrow L = \begin{array}{c} \begin{matrix} 1 & y & z & s_1 & s_2 \end{matrix} \\ \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \\ 5 & 0 & 1 & 0 & 1 \end{pmatrix} \end{array}$$

The same method gives:

$$R = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 \end{pmatrix} \qquad \text{and} \qquad O = \begin{pmatrix} 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{pmatrix}$$

Thus, given the public statement $\mathbf{x} = (1, y)$, the Prover wants to convince the Verifier that the Prover knows $\mathbf{w} = (z, s_1, s_2)$ such that $\mathbf{a} = (\mathbf{x}, \mathbf{w})$ satisfies

$$L\mathbf{a} \circ R\mathbf{a} = O\mathbf{a}$$

where $\circ$ is the entrywise (or Hadamard) product.

This illustrative example leads us to the following definition:

Definition 3.1. A *rank-1 constraint system* (R1CS) over a (finite) field $\mathbb{F}$ is defined by three matrices $L, R, O \in \mathcal{M}_{N \times M}(\mathbb{F})$. A vector $\mathbf{a} \in \mathbb{F}^M$ satisfies the system if

$$L\mathbf{a} \circ R\mathbf{a} = O\mathbf{a}.$$

If we split $\mathbf{a} = (\mathbf{x}, \mathbf{w})$ into a public statement $\mathbf{x} \in \mathbb{F}^\ell$ and a private witness $\mathbf{w} \in \mathbb{F}^{M-\ell}$, we can define a natural relation: for $\mathsf{x} = ((L, R, O), \mathbf{x})$ and $\mathsf{w} = \mathbf{w}$, we have $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}$ if and only if $\mathbf{a} = (\mathbf{x}, \mathbf{w})$ satisfies the R1CS for (L, R, O). M and N are respectively called the number of *wires* and *constraints*.

These matrices depend only on the fixed circuit (i.e., program) and must therefore be known by the Verifier. As noted above, if the Verifier is expected to verify one and only one program, this data can be hard-coded and requires little RAM. Nevertheless, the model considered here is a flexible Verifier that can check proofs for various programs. Therefore, these objects must be stored and loaded on the Verifier device: this is what we have called the *Verifier artifacts*.

Let us naively estimate the size of these matrices on an example. A SHA256 implementation for a 64-byte input has around 30000 constraints (see [SG23]); moreover, the numbers of constraints and wires are often of the same order in practice. Therefore, for a 32-bit field, the size of the artifacts is around

$$N \cdot M \cdot \frac{3\log_2(|\mathbb{F}|)}{8} \text{ B} = 12 \times 30000^2 \text{ B} \approx 10 \text{ GB}$$

Of course, if these matrices were loaded in this form, they would be unusable on most devices, let alone on a low-resource one. In practice, these matrices are sparse, which makes a lighter representation possible. For each of L, R, and O, one could use three lists row, col, and values, which collect only the non-zero coefficients: for an integer k, $\mathsf{values}[k]$ is the value of the matrix at coordinates $(\mathsf{row}[k], \mathsf{col}[k])$. If the three matrices contain S such coefficients, then the required storage is approximately $S(\log_2(|\mathbb{F}|) + \log_2(N) + \log_2(M))$ bits, with $S \ll N \cdot M$.

3.2 AIR: Algebraic Intermediate Representation

Another approach to arithmetization represents an execution by means of a trace. Although AIR does not inherently require a virtual machine, it is often used with a dedicated virtual machine, called a *zkVM*, that executes the program while producing an execution trace. In this setting, the trace describes not only the execution of the program, but also the internal machinery of the virtual machine. Thus, one proves the correct execution of the whole procedure carried out by the zkVM.

The idea of an *Algebraic Intermediate Representation* (AIR) is to represent an execution by a *trace*, that is, an array of values, and then to impose *algebraic constraints* on this trace. In the setting of SNARKs, these constraints are defined over a finite field $\mathbb{F}$. In the zkVM setting, generating this trace and its associated constraints is complex and specific to each zkVM. For this reason, we restrict ourselves to a simple illustrative example.

Consider a program that computes the N-th term of the Fibonacci sequence, defined by $F_1 = F_2 = 1$ and $F_{n+2} = F_{n+1} + F_n$. At each step, the state of the computation is described by two registers, denoted a and b, which respectively contain the current and next terms. Row i of the trace will therefore encode the pair $(a_i, b_i) = (F_{i+1}, F_{i+2})$.

Algorithm 1 Iterative computation of the Fibonacci sequence

Require: $N \geq 2$
1: $a \leftarrow 1$
2: $b \leftarrow 1$
3: **for** $i = 0$ **to** $N-2$ **do**
4: record the trace row $(a_i, b_i) = (a, b)$
5: $(a, b) \leftarrow (b, a+b)$
6: **end for**
7: record the last row $(a_{N-1}, b_{N-1}) = (a, b)$
8: **return** a

We thus obtain a two-column execution trace. For $N = 4$, it has the following form:

Row i	a_i	b_i
0	1	1
1	1	2
2	2	3
3	3	5

Table 2: Execution trace for the Fibonacci example.

The AIR construction then consists of translating the update $(a,b) \mapsto (b,a+b)$ into algebraic constraints between two consecutive rows. For every $0 \leq i \leq N-2$, we impose

$$a_{i+1} - b_i = 0, \qquad b_{i+1} - (a_i + b_i) = 0.$$

To these transition constraints, we add boundary constraints to fix the initial state and, possibly, a final constraint to impose the public output.

Type	Constraint	Interpretation
Initial boundary	$a_0 = 1$	fixes the first term of the sequence
Initial boundary	$b_0 = 1$	fixes the second term of the sequence
Transition	$a_{i+1} - b_i = 0$	the left register receives the previous value of the right one
Transition	$b_{i+1} - (a_i + b_i) = 0$	the right register contains the sum of the two previous terms
Final boundary (optional)	$a_{N-1} = y$	imposes that the public output is $y = F_N$

Table 3: AIR constraints for the Fibonacci example.

These relations can naturally be reformulated as *constraint polynomials*. If we denote by (x_1,x_2) the current row of the trace and by (y_1,y_2) the next row, then the transition constraints of the Fibonacci example are given by the two polynomials

$$P_1(x_1,x_2,y_1,y_2) = y_1 - x_2, \qquad P_2(x_1,x_2,y_1,y_2) = y_2 - (x_1 + x_2).$$

A valid trace then satisfies, for every pair of consecutive rows $\big((a_i,b_i),(a_{i+1},b_{i+1})\big)$,

$$P_1(a_i,b_i,a_{i+1},b_{i+1}) = 0, \qquad P_2(a_i,b_i,a_{i+1},b_{i+1}) = 0.$$

Similarly, the initial conditions can be viewed as boundary polynomials:

$$B_1(x_1,x_2) = x_1 - 1, \qquad B_2(x_1,x_2) = x_2 - 1,$$

which must vanish on the first row of the trace. Finally, if one wants to impose a public output $y = F_N$, one may add the final polynomial, which must vanish on the last row:

$$B_f(x_1,x_2) = x_1 - y.$$

The trace is indexed not directly by integers, but by the elements of a cyclic multiplicative subgroup $G = \langle g \rangle \subset \mathbb{F}^*$, called the *trace domain*. This group has cardinality at least N and permits the use of FFTs for efficient interpolation. In general, one chooses g as a 2^r-th root of unity, where r is the smallest integer such that $2^r \geq N$, and pads the trace. One may then take $N = 2^r$. Thus, the i-th row of the trace is associated with the point $g^i \in G$.

It remains to interpolate each column into a polynomial of degree $< N$, called a *trace polynomial*, representing the corresponding register. In the Fibonacci example, we have $A, B \in \mathbb{F}^{<N}[X]$, such that

$$A(g^i) = a_i, \qquad B(g^i) = b_i, \qquad 0 \leq i \leq N-1.$$

Passing to the next row is then expressed by the multiplicative shift $X \mapsto gX$. The transition constraints are reformulated as polynomial relations evaluated on a subset of the domain:

$$A(gX) - B(X) = 0, \qquad B(gX) - A(X) - B(X) = 0.$$

Remark. In this brief presentation, we restrict ourselves to traces with transition constraints of order 1, in the sense that these constraints involve only two successive steps. The papers [BSGKS19] and [MF23] use the notion of a *mask*, which makes it possible to generalize AIR to higher-order transitions.

Building on this example, we can now present a general definition of AIR, following the approach of [Hab22]. The preceding illustrative presentation can be summarized by the following figure:

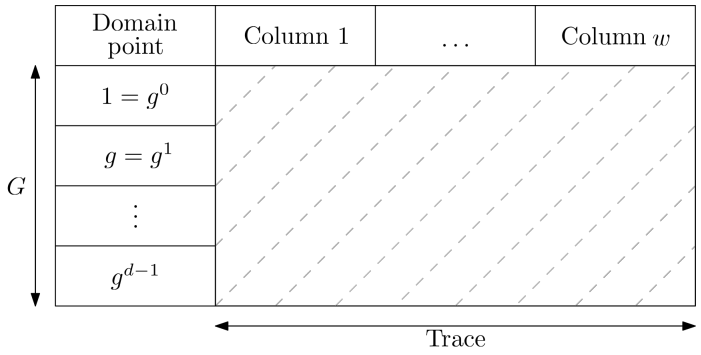


Figure 7: Trace representation

Definition 3.2 (AIR Relation)**.** We define the relation $\mathcal{R}_{\text{AIR}}$ by the following data and conditions:

1. **Instance format.** We set $\mathsf{x} = (\mathbb{F}, g, w, \mathsf{Cst})$, where
   - $\mathbb{F}$ is a finite field;
   - g is an element of the multiplicative group $\mathbb{F}^*$ of order d. We denote $G := \langle g \rangle$;
   - w is a strictly positive integer;
   - Cst is a collection of $c \geq 1$ constraints, specified by:
     (i) $(H_1, \ldots, H_c)$ cosets of subgroups of G, contained in G: for every $i \in \{1, \ldots, c\}$, there exist $G_i \leq G$ and $a_i \in G$ such that $H_i = a_i \cdot G_i$. This coset is said to be non-trivial if $a_i \notin G_i$.
     (ii) $(P_1, \cdots, P_c)$ polynomials in $\mathbb{F}[X_1, \ldots, X_w, Y_1, \ldots, Y_w]$.
2. **Witness format.** The witness w is a w-tuple $(f_1, \ldots, f_w)$ of polynomials in $\mathbb{F}[X]$.
3. **Relation.** We have $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\text{AIR}}$ if and only if:
   - for every $j \in \{1, \ldots, w\}$, we have $\deg(f_j) < d$;
   - for every $i \in \{1, \ldots, c\}$ and every $x \in H_i$, we have

$$P_i(f_1(x), \ldots, f_w(x), f_1(g \cdot x), \ldots, f_w(g \cdot x)) = 0.$$

As for the R1CS arithmetization, let us naively estimate the size of the AIR artifacts and how they are handled in practice. The main part of the Verifier artifacts comes from the AIR constraint polynomials. For the OpenVM SHA256-specific AIR used in our experiments, we measured approximately 1000 constraint polynomials in 1000 variables, each of degree at most 3. Moreover, this proof system is built

on the BabyBear field, whose elements are 4 bytes long. Thus, if all these polynomials were stored fully and naively, they would require approximately:

$$1000 \times \binom{1003}{3} \times 4 \text{ B} > 4000 \times 160000000 \text{ B} > 500 \text{ GB}.$$

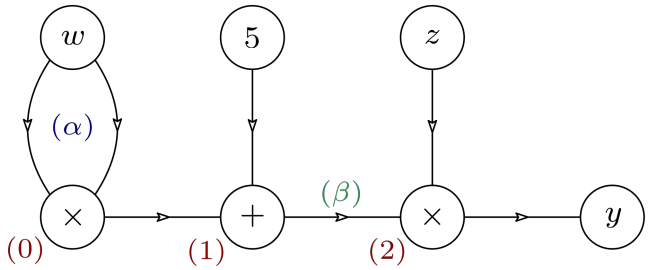
Fortunately, as in R1CS, these polynomials are sparse, and it is sufficient to store their non-zero coefficients, which greatly reduces the required space. A simple representation of $P(X) \in \mathbb{F}^{\leq 3}[X_1, \ldots, X_s]$ uses pairs (c, α), where $c \in \mathbb{F}^*$ and $\alpha = (\alpha_1, \ldots, \alpha_s)$, with $0 \leq \alpha_i \leq 3$. Thus, if we have L such non-zero coefficients and store each coordinate of α in one byte[4], we would need approximately $L(\frac{\log_2(|\mathbb{F}|)}{8} + s)$ bytes.

Remark. In reality, OpenVM does not use this representation, but a directed acyclic graph representation, which is much more efficient for this proof system. Nevertheless, the idea remains the same, as it reduces the size of Verifier artifacts by taking into account the sparsity of the polynomials.

3.3 Plonkish

The Plonkish arithmetization, introduced in the Plonk [GWC19] proof system, can be seen as a hybrid of the R1CS and AIR arithmetizations. As in R1CS, the program is described by a circuit. Polynomials then encode the logic and structure of that circuit; they form the core of the Verifier artifacts for this arithmetization. Finally, the Prover constructs a trace from the circuit, from which the final proof is derived. As before, let us work with an illustrative example to see how this technique works.

Let us consider a program f that takes z as a public input and w as a private input, and computes $f(z,w) = zw^2 + 5z$. The following circuit describes f:



Thus, an honest Prover who evaluates f on $(z,w) = (4,3)$ can build the following trace using the left and right inputs and the output of each gate:

Inputs	$z = 4$ $w = 3$		
Gate 0	3	3	9
Gate 1	5	9	14
Gate 2	4	14	56

Table 4: Plonkish trace example

The trace is indexed by a cyclic group $G = \langle g \rangle$[5]. For a simpler description, assume that $|G| = 11$ in our example, so we can use g^{-1} and g^{-2} to identify the two inputs, and g^{3i}, g^{3i+1}, and g^{3i+2} to

[4]Constraint polynomials are very often of low degree, so it is a very mild assumption to suppose this degree to be lower than 256 in general.

[5]As in the AIR arithmetization, this group must be FFT-friendly since it will be used to perform interpolation.

respectively identify the first, second, and third columns of the trace, where $i \in \{0,1,2\}$. Similarly to the AIR arithmetization, we can interpolate this description into a polynomial P that describes the entire trace. This polynomial is then used to prove the correct execution of the program, since it must satisfy characteristic properties. Some of them are related to the inputs and outputs, but we focus on those that describe the structure of our circuit:

1. Each gate enforces the intended operation. For instance, this leads to the following equations:

$$(1) \ : \ P(g^2) = P(g^0) \times P(g^1) \qquad (2) \ : \ P(g^5) = P(g^3) + P(g^4).$$

2. The copy constraints induced by the wiring are satisfied. This implies checking equalities between some gate inputs and outputs. For instance, two such equalities are:

$$(\alpha) \ : \ P(g^0) = P(g^1) = P(g^{-2}) \qquad (\beta) \ : \ P(g^5) = P(g^7)$$

For the first property, we use the following fundamental equation for $x = g^{3i}$:

$$q_M(x)P(x)P(x\cdot g) + q_L(x)P(x) + q_R(x)P(x\cdot g) + q_O(x)P(x\cdot g^2) + q_C(x) = 0 \qquad (\mathcal{G})$$

This equation allows us to describe the common gates used in our circuit:

- **Addition gate.** Let us set $q_L(x) = q_R(x) = 1$, $q_O(x) = -1$ and $q_M(x) = q_C(x) = 0$; we have
$$P(x) + P(x\cdot g) - P(x\cdot g^2) = 0$$
- **Multiplication gate.** Let us set $q_L(x) = q_R(x) = q_C(x) = 0$, $q_M(x) = 1$ and $q_O(x) = -1$; we have
$$P(x)P(x\cdot g) - P(x\cdot g^2) = 0$$
- **Constant addition gate.** Let us set $q_L(x) = 1$, $q_R(x) = q_M(x) = 0$, $q_O(x) = -1$ and $q_C(x) = \mu \in \mathbb{F}$; we have
$$P(x) - P(x\cdot g^2) + \mu = 0$$
- **Constant multiplication gate.** Let us set $q_L(x) = \mu \in \mathbb{F}$, $q_R(x) = q_M(x) = q_C(x) = 0$ and $q_O(x) = -1$; we have
$$\mu P(x) - P(x\cdot g^2) = 0$$

It is important to note that this description allows more flexibility in the gates that can be used, as long as they can be described by Equation $\mathcal{G}$. Therefore, optimizations in the circuit description can reduce the number of gates and hence the size of artifacts (and sometimes proofs).

Let us now consider the second property. The main idea is to define a partition of G whose blocks collect the positions at which the values must be equal. We then define $\sigma \in \mathfrak{S}(G)$ as a composition of cycles on the blocks of this partition. For instance, we obtain the following cycles:

$$(\alpha) \ \to \ (g^0 \ \ g^1 \ \ g^{-2}) \qquad (\beta) \ \to \ (g^5 \ \ g^7)$$

We can now interpolate a polynomial W such that, for every $x \in G$, $W(x) = \sigma(x)$. Finally, our trace polynomial P must satisfy:

$$\forall x \in G, \quad P(x) = P(W(x)). \qquad (\mathcal{P})$$

This description allows us to define a relation associated with the Plonkish arithmetization:

Definition 3.3 (Plonkish Relation)**.** For an ℓ-gate circuit with s public inputs and t private inputs, we define the relation $\mathcal{R}_{\text{Plonkish}}$ by the following data and conditions:

1. **Instance format.** We set $\mathsf{x} = (\mathbb{F}, G, \mathbf{y}, q_M, q_L, q_R, q_O, q_C, W)$ with
   - $\mathbb{F}$ is a finite field;
   - $G = \langle g \rangle \leq \mathbb{F}^*$ is a cyclic group of order $d = 3\ell + r$, where $r = s + t$;
   - $\mathbf{y} = (y_1, \ldots, y_s) \in \mathbb{F}^s$;
   - q_M, q_L, q_R, q_O, q_C are polynomials in $\mathbb{F}^{<d}[X]$;
   - $W \in \mathbb{F}^{<d}[X]$ is a G-permutation polynomial.
2. **Witness format.** The witness w is a polynomial P in $\mathbb{F}[X]$.
3. **Relation.** We have $(\mathsf{x}, \mathsf{w}) \in \mathcal{R}_{\text{Plonkish}}$ if and only if:
   - $\deg(P) < d$;
   - for every $j \in \{1, \ldots, s\}$, we have $P(g^{-j}) = y_j$;
   - for every $i \in \{0, \ldots, \ell - 1\}$, setting $x = g^{3i}$, P satisfies Equation ($\mathcal{G}$):
   $$q_M(x)P(x)P(x \cdot g) + q_L(x)P(x) + q_R(x)P(x \cdot g) + q_O(x)P(x \cdot g^2) + q_C(x) = 0;$$
   - for every $x \in G$, P satisfies Equation ($\mathcal{P}$):
   $$P(x) = P(W(x)).$$

Remark. What we call *Plonkish* arithmetization here may differ from other descriptions. The idea remains the same, as we describe the structure of the circuit using gate and permutation polynomials; nevertheless, there are various ways of doing this depending on the gates considered and the protocol approach.

Here we see that the size of the Verifier artifacts is mainly determined by the six polynomials describing the gates and the wiring. Naively, if these polynomials were stored as dense coefficient lists, they would require

$$6d \cdot \frac{\log_2(|\mathbb{F}|)}{8} \text{ B}.$$

For a rough comparison, let us take a SHA256 circuit with approximately $\ell = 30000$ gates[6], and neglect the few input wires so that $d \simeq 3\ell$. Over a 32-bit field, this gives

$$6 \times 90000 \times 4 \text{ B} \approx 2 \text{ MB}.$$

This is much smaller than the previous naive estimates because these artifacts are univariate polynomials, so their dense representation grows linearly with the number of gates. In practice, these polynomials are not always stored in this dense coefficient form. The selector polynomials q_M, q_L, q_R, q_O, and q_C are often highly structured on the evaluation domain, since many rows use the same few gate types, and they can be represented by selector vectors, gate descriptions, or compressed lists of non-zero values. The wiring polynomial W is generally less sparse as a polynomial, but it comes from a permutation, so implementations usually store this permutation or its evaluations rather than arbitrary coefficients. Thus, the effective size of Plonkish artifacts strongly depends on the chosen representation.

This concludes the presentation of the main existing arithmetization families. There exist many variations of all of them and other proof systems with their own arithmetization. It is also interesting to note that Setty, Thaler, and Wahby defined in 2023 a generalization of these arithmetizations called *Customizable Constraint Systems* (CCS) [STW23]. Their approach makes it possible to group the three preceding approaches under a single definition and create new ones after specifying some parameters.

[6] We take the same number of gates as for R1CS, although the availability of more gate types can reduce this number.

3.4 Reducing Verifier artifact size

3.4.1 Commitment-based preprocessing

At a basic level, this technique consists of replacing large Verifier artifacts with short commitments, produced during a preprocessing step and later hard-coded or loaded by the Verifier. These commitments bind the proof to the proven circuit or program and authenticate computations that, in a naive verification procedure, would require the Verifier to read or process the corresponding polynomials. This is exactly the kind of guarantee provided by a Polynomial Commitment Scheme (PCS): the Prover can work with the committed polynomials, while the Verifier only checks openings or evaluation claims. The price is additional Prover work and possibly larger proofs, but the program-specific artifacts can become much smaller.
A classical setting in which this mechanism is particularly visible is an elliptic curve endowed with a pairing, namely the KZG-PCS setting (see [KZG10]). This setting is an important approach to SNARK construction, including in [Gro16] and [GWC19]. We present it as a first example of commitment-based preprocessing:

For simplicity, let us fix a cyclic additive subgroup $G = \langle g \rangle$ of prime order p of an elliptic curve, with scalar field $\mathbb{F} = \mathbb{F}_p$, and a symmetric pairing function $e : G \times G \longrightarrow G_T$. Then, the authority samples $\tau \leftarrow\$ \mathbb{F}^*$ and publishes a *Structured Reference String*:

$$\mathsf{srs} = (h_0, \ldots, h_d) = (g, \tau \cdot g, \ldots, \tau^d \cdot g) \in G^{d+1}$$

where $d < p$ is the maximum supported degree. The Prover uses this string to compute commitments; the Verifier only needs the relevant commitments and the few group elements involved in the opening check. Thus, the first observation is the following property:

Proposition 3.1. *If* $f = a_0 + a_1 X + \cdots + a_d X^d \in \mathbb{F}^{\leq d}[X]$, *then*

$$a_0 \cdot h_0 + a_1 \cdot h_1 + \cdots + a_d \cdot h_d = f(\tau) \cdot g$$

In the following, we will write $\mathsf{com}(f) = f(\tau) \cdot g$. It is important to notice that the points of G can be published without revealing the value of τ. Now, let us recall the idea behind the KZG-PCS:

0. An honest Prover knows $f = a_0 + a_1 X + \cdots + a_d X^d \in \mathbb{F}^{\leq d}[X]$ such that $f(z) = y$, with $z, y \in \mathbb{F}$, and wants to convince the Verifier of this fact.
1. The Prover sends $\mathsf{com}(f)$, then computes
$$q(X) = \frac{f(X) - y}{X - z} \in \mathbb{F}^{\leq d-1}[X]$$
and sends $\mathsf{com}(q)$, z and y to the Verifier.
2. The Verifier must check that $\mathrm{e}(\mathsf{com}(q), h_1 - z \cdot g)\mathrm{e}(g,g)^y = \mathrm{e}(\mathsf{com}(f), g)$.

The first idea would be to directly check that $(\tau - z) \cdot \mathsf{com}(q) = \mathsf{com}(f) - y \cdot g$, but the Verifier does not know the value of τ and therefore cannot compute that point. It is for that exact reason that the pairing function is needed. The specific properties of such a function allow us to compute everything we need. More precisely, we have:

$$\begin{aligned}
\mathrm{e}(\mathsf{com}(q), h_1 - z \cdot g)\mathrm{e}(g,g)^y &= \mathrm{e}(q(\tau) \cdot g, (\tau - z) \cdot g)\mathrm{e}(g,g)^y \\
&= \mathrm{e}(g,g)^{q(\tau)(\tau - z) + y} \\
&= \mathrm{e}(g,g)^{f(\tau)} \qquad \text{since } f(X) = q(X)(X - z) + y \\
&= \mathrm{e}(f(\tau) \cdot g, g) \\
&= \mathrm{e}(\mathsf{com}(f), g)
\end{aligned}$$

Remark. For our purposes, the important point is that the Prover's claim is tied to a polynomial f. However, one can imagine that this polynomial is related to the program itself, such as the selector polynomials and W in Section 3.3. In that case, the commitment $\mathsf{com}(f)$ can be provided during preprocessing, thereby reducing the size of Verifier artifacts: a large polynomial can be wrapped as a single element of G.
The binding security of the original KZG construction relies on pairing-based assumptions, notably a variant of the Strong Diffie–Hellman assumption, rather than on the discrete logarithm assumption alone. Like other constructions based on elliptic-curve pairings, it is not post-quantum. Moreover, security requires that τ remain unknown: if the Prover learns its value, he can construct false openings. The generation of this secret is called a *trusted setup*, which is precisely what *transparent* proof systems avoid.

This KZG-based example should be understood only as a first approach to commitment-based preprocessing. There exist many other PCS used in proof systems, with different assumptions and different effects on proof size, setup requirements, and Verifier work. In particular, transparent and post-quantum systems often rely instead on hash-based commitments combined with FRI; Appendix A gives a more detailed presentation of a FRI-based PCS, which is the relevant family for the Plonky2 prover considered later. In many practical proof systems, a PCS primarily serves to reduce proof size, while the Verifier still needs circuit-specific data, such as polynomial commitments, evaluations, or verifying-key elements, to authenticate the program. The exact form and size of this data depend on the protocol. Thus, some proof systems have a very fast Verifier while still requiring it to load sizeable program-specific objects. Finally, it is important to note that such techniques can involve a substantial trade-off by making proofs much larger (see Appendix B). This can be a significant obstacle for a practical implementation, even when combined with the streaming technique shown in Section 4.3.

3.4.2 Recursion and universal proving

Recursive systems are often considered to better control proof size or to batch a large number of proofs into a single proof, which is then verified. They can also substantially reduce the size of program-specific artifacts stored in the Verifier device. Let $\pi_{\mathcal{P}}$ be a proof of the execution of a program $\mathcal{P}$. Rather than verifying $\pi_{\mathcal{P}}$ directly, the Prover proves that a fixed verifier program $\mathcal{V}$ accepts $\pi_{\mathcal{P}}$ with a statement containing the program identifier and the relevant public inputs and outputs. The external Verifier then checks only this outer proof against the fixed artifacts of $\mathcal{V}$ as described in Figure 8

Consequently, the program-specific information can be reduced to a short *digest*, for instance a hash of $\mathcal{P}$, provided that the external Verifier compares it with the expected identifier. The artifacts of $\mathcal{V}$ are independent of $\mathcal{P}$[7] and can therefore be hard-coded in the device. This procedure is summarized in the following scheme:

Universal proving, often implemented through a zkVM, has a similar effect on Verifier artifacts but is distinct from recursion. Here, the fixed program is a virtual machine $\mathcal{U}$, and the program $\mathcal{P}$ is provided as an input to $\mathcal{U}$. The Prover proves that $\mathcal{U}$ executed $\mathcal{P}$ on the claimed inputs and produced the claimed outputs. Since the arithmetization describes the fixed virtual machine rather than each program itself, the corresponding Verifier artifacts can again be hard-coded. The external Verifier only needs an expected program identifier to bind the proof to $\mathcal{P}$.

Universal proving does not inherently require recursion, although the two techniques are frequently combined: a zkVM can use recursion to aggregate or compress proofs of its execution. In both cases, the trade-off is that the fixed verifier or virtual-machine program is complex to arithmetize and optimize. Thus, these approaches replace potentially large, variable program-specific artifacts with fixed artifacts embedded in the Verifier implementation and a short, program-specific identifier.

[7] In practice, the verifier program $\mathcal{V}$ supports only a specified range of proof systems and proof sizes, which influences the final proof and artifact sizes.

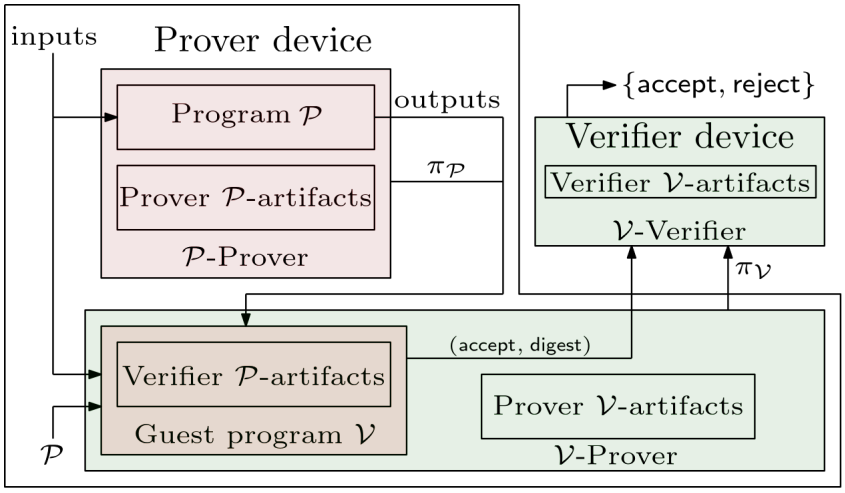


Figure 8: Recursive proof systems

3.4.3 Integrity-check streaming

Another approach does not reduce artifact size itself, but rather makes it less relevant. It applies in either of the following two cases:

- the Verifier artifacts are sent directly with the proof by the Prover;
- we use a hybrid device that can store Verifier artifacts in an untrusted storage area.

In both cases, the Secure Element only performs the Verifier computations using the proof and the artifacts. The streaming approach, either from the Prover or directly from the device itself, allows the verification data to be processed piece by piece. One could split the artifacts into chunks $A = C_1 \| \ldots \| C_n$ that are small enough for the Secure Element and send them piece by piece. However, with this design, an attacker could have corrupted the artifacts beforehand since they are not stored in trusted memory. A solution to that threat is to use the chain-hash method, which checks the integrity of each chunk:

1. A trusted authority computes $h_n := H(n)$ and $h_i = H(C_{i+1} \| h_{i+1})$ for all $i \in \{0, \ldots, n-1\}$ and sends h_0 to the Verifier;
2. The Verifier sets $\widehat{h}_0 := h_0$. At each step $i \in \{1, \ldots, n\}$, it receives a candidate pair $\widetilde{C}_i$ and $\widetilde{h}_i$, checks $\widehat{h}_{i-1} = H(\widetilde{C}_i \| \widetilde{h}_i)$, and, if the check succeeds, sets $\widehat{h}_i := \widetilde{h}_i$ for the next step.

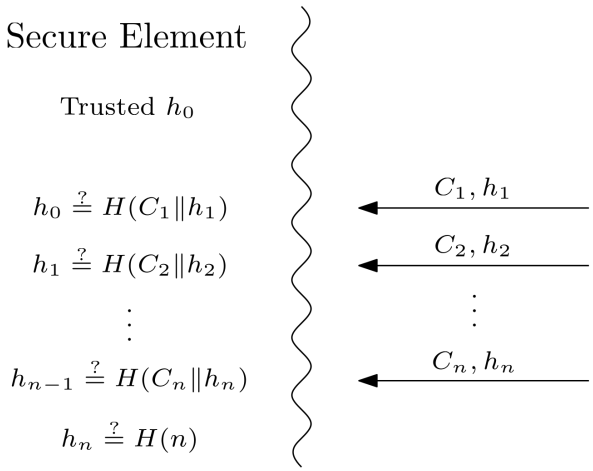


Figure 9: Authenticated storage design

At each step, the Verifier must carry out the required computations using the artifact chunk it has just received. Therefore, this design requires these chunks to be used only once and to be independent of one another, which is not always the case. In addition, the limited streaming bandwidth of the device can impose a substantial cost, since streaming is not among the fastest operations on Secure Elements. Nevertheless, considering only memory constraints, it offers a safe alternative:

Proposition 3.2. *If H is collision resistant and all the chunks C_i have the same fixed size, then, except with negligible probability, the previous protocol accepts only chunks $\widetilde{C}_i$ received by the Verifier such that $\widetilde{C}_i = C_i$ for all $i \in \{1, \dots, n\}$.*

Proof. We prove the statement by induction on i. Initially, $\widehat{h}_0 = h_0$ is the authenticated value received from the trusted authority. Suppose that, before step $i \in \{1, \dots, n\}$, we have $\widehat{h}_{i-1} = h_{i-1}$. If the Verifier accepts the candidate pair $\widetilde{C}_i$ and $\widetilde{h}_i$, then

$$H(\widetilde{C}_i \,\|\, \widetilde{h}_i) = \widehat{h}_{i-1} = h_{i-1} = H(C_i \,\|\, h_i).$$

Since H is collision resistant, this implies, except with negligible probability, that

$$\widetilde{C}_i \,\|\, \widetilde{h}_i = C_i \,\|\, h_i.$$

Since the chunks have the same fixed size and hash outputs have fixed size, this concatenation has a unique parsing, hence $\widetilde{C}_i = C_i$ and $\widetilde{h}_i = h_i$. Therefore $\widehat{h}_i = \widetilde{h}_i = h_i$, so the induction invariant is preserved. It follows that every accepted chunk is equal to the corresponding original chunk. □

Even if the Verifier artifact size is reduced to a single hash value, all chunks must still be streamed to the Verifier, either by the Prover device or from untrusted storage. In our naive model, the Verifier can use the chunk data linearly during the verification phase. Finally, we must account for the cost of the hash values accompanying the chunks. This part is mostly negligible: if the ratio between the artifact size and the chunk size remains below 1000, then even in this very worst case, we would need to add around 32 kB to the final proof, which is negligible compared with common proof sizes.

To conclude, there exist many solutions to reduce Verifier artifact size in existing proof systems; however, it is important to keep in mind that all of them come with obstacles. They may affect Prover efficiency, security, or programming flexibility. Moreover, most of these procedures are not designed primarily to reduce Verifier artifacts, but to address other considerations that are more prominent in SNARK research, such as Prover and Verifier speed and proof size. For instance, Flock [BRW26] is built to prove multiple calls of the same function efficiently. As a consequence, the Verifier artifacts are almost invariant with respect to the number of executions, which gives a strong asymptotic improvement. However, for a single call, in our measurements, the Flock proof system has one of the largest artifact sizes among those we tested.

3.5 Comparison of artifacts sizes

As explained previously, Verifier artifact size depends mainly on the arithmetization, the protocol configuration, and the chosen artifact representation. The obtained results are represented in the Figure 10.

First, as expected, SP1 and RISC Zero have the smallest artifacts. Both proof systems are benchmarked in recursive, compressed modes, so their external artifacts contain only a hash and/or metadata of the program $\mathcal{P}$. Next come Circom, Barretenberg, and Plonky2, which use preprocessing techniques. Plonky2 uses a post-quantum, FRI-based preprocessing approach (described in appendix A); in these measurements, its proofs are larger than those of Circom or Barretenberg. The remaining proof systems do not use the previously mentioned techniques to reduce the size of their artifacts. Among these *raw* implementations of Verifier artifacts, the AIR implementation (OpenVM) is much smaller than the R1CS implementations (Binius64, Flock, and Spartan2).

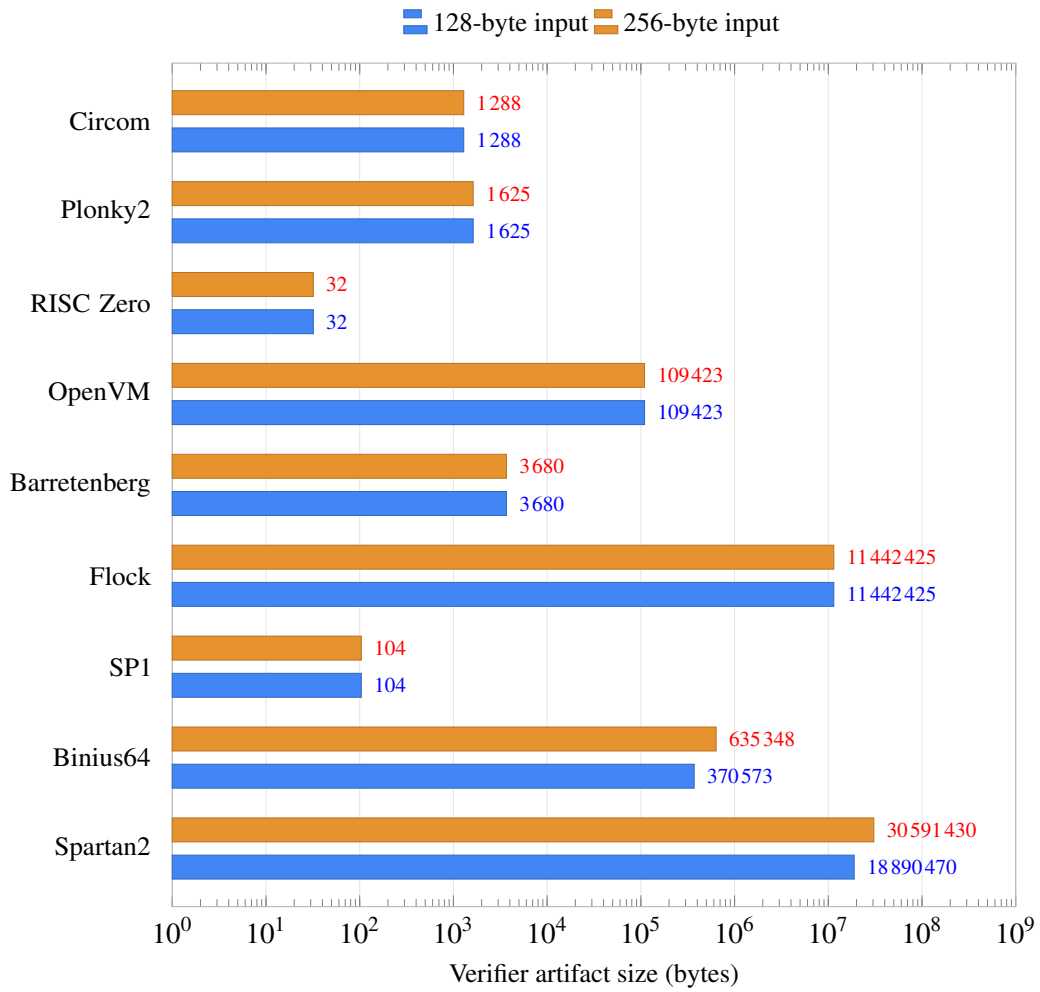


Figure 10: Verifier artifact sizes for SHA-256 programs

4 Proof constraints

4.1 A simplified proof-size estimate for FRI

In this section, we give a quick description of the *Fast Reed-Solomon Interactive Oracle Proofs of Proximity* protocol (FRI), which is used by several proof systems, such as RISC Zero, Plonky2, and Binius64. The goal is to illustrate the kind of information contained in a proof and the impact of the chosen proof system on these data. Since SNARK constructions are not the main goal of this report, many steps will be voluntarily omitted. It is therefore recommended to read Appendix A or the side report [Mar26] for a more precise understanding.

To explain how FRI can be used, we consider the STARK [BSBHR18b] approach used in RISC Zero. However, we first recall some important notions used here:

Definition 4.1. Consider a finite field $\mathbb{F}$, a domain $\mathcal{L} \subset \mathbb{F}$, and an integer $d > 0$. The associated Reed-Solomon code is defined by:

$$\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d] = \{u : \mathcal{L} \longrightarrow \mathbb{F},\ \exists f \in \mathbb{F}^{<d}[X],\ f|_{\mathcal{L}} = u\}.$$

We can equivalently define an RS code by using the rate $\rho \in]0,1[$ defined by $\rho := d/|\mathcal{L}|$. Also, for an oracle $u : \mathcal{L} \longrightarrow \mathbb{F}$, we define the distance between u and $\mathcal{C}$ by using the Hamming distance:

$$\Delta(u, \mathcal{C}) := \min_{v \in \mathcal{C}} \Delta(u, v) \qquad \text{where} \qquad \Delta(u, v) = \frac{|\{x \in \mathcal{L}, u(x) \neq v(x)\}|}{|\mathcal{L}|} \in [0, 1].$$

Let us now present the three main components of a STARK proof system:

- AIR arithmetization, which is described in Section 3.2;
- the *Algebraic Linking IOP* protocol (ALI), which reduces verification of an AIR witness to consistency and proximity claims involving several oracles;
- the FRI protocol, which allows a Prover to efficiently convince a Verifier that an oracle is sufficiently close to a fixed RS code.

The FRI protocol consists of two phases: the COMMIT phase, during which the Prover commits to oracles linked by important relations, and the QUERY phase, during which the Verifier queries these oracles at points of $\mathcal{L}$ with high reliability[8] to check these relations. We denote by r the number of COMMIT rounds and by t the number of query points in the QUERY phase.

After omitting the ALI protocol, the BCS compilation, Merkle authentication paths, and the final FRI object, we obtain the following lower bound for the data processed by an idealized FRI transcript for one oracle:

- COMMIT: $r+1$ oracle commitments and r challenges;
- QUERY: t query points and $3 \times t \times r$ answers to queries.

The short security analysis provided in Appendix A and in [Mar26] gives the following result: for $r = 12$, a security parameter $\lambda = 128$, a rate $\rho = 2^{-5}$, and a 192-bit field $\mathbb{F}$, we choose $t \approx 50$ for this simplified estimate. Also, an element of $\mathbb{F}$ occupies 24 bytes and an oracle commitment occupies 32 bytes[9]. It follows:

- COMMIT: $13 \times 32\ \text{B} + 12 \times 24\ \text{B} = 704\ \text{B}$;
- QUERY: $50 \times 24\ \text{B} + 3 \times 12 \times 50 \times 24\ \text{B} = 44400\ \text{B}$.

This calculation is deliberately a lower bound, rather than a prediction of the size of a concrete proof. In particular, it omits the Merkle authentication paths, the final FRI object, the data introduced by ALI and the BCS transformation, and the fact that a STARK proof generally involves several oracles. These factors explain why concrete proofs can reach hundreds of kB, or even some MB.

4.2 Proof-size analysis

We presented some components that can appear in a FRI-based proof, here in a simplified STARK setting, in order to explain why some proof systems can reach hundreds of kB. Let us now present in the figure 11 the results obtained by measuring proof sizes for the SHA256 program.

For a device with only a few dozen kB of RAM, the first conclusion would be that only non-post-quantum proof systems can be supported if the entire proof must be loaded at once. However, the next section shows that, under a precise analysis of the proof structure, it is possible for the Verifier to avoid loading the whole proof at once.

[8] This follows from the Merkle commitments described in [Mar26], and more precisely in [CY24].

[9] We use a 256-bit hash output, which targets 128-bit collision resistance.

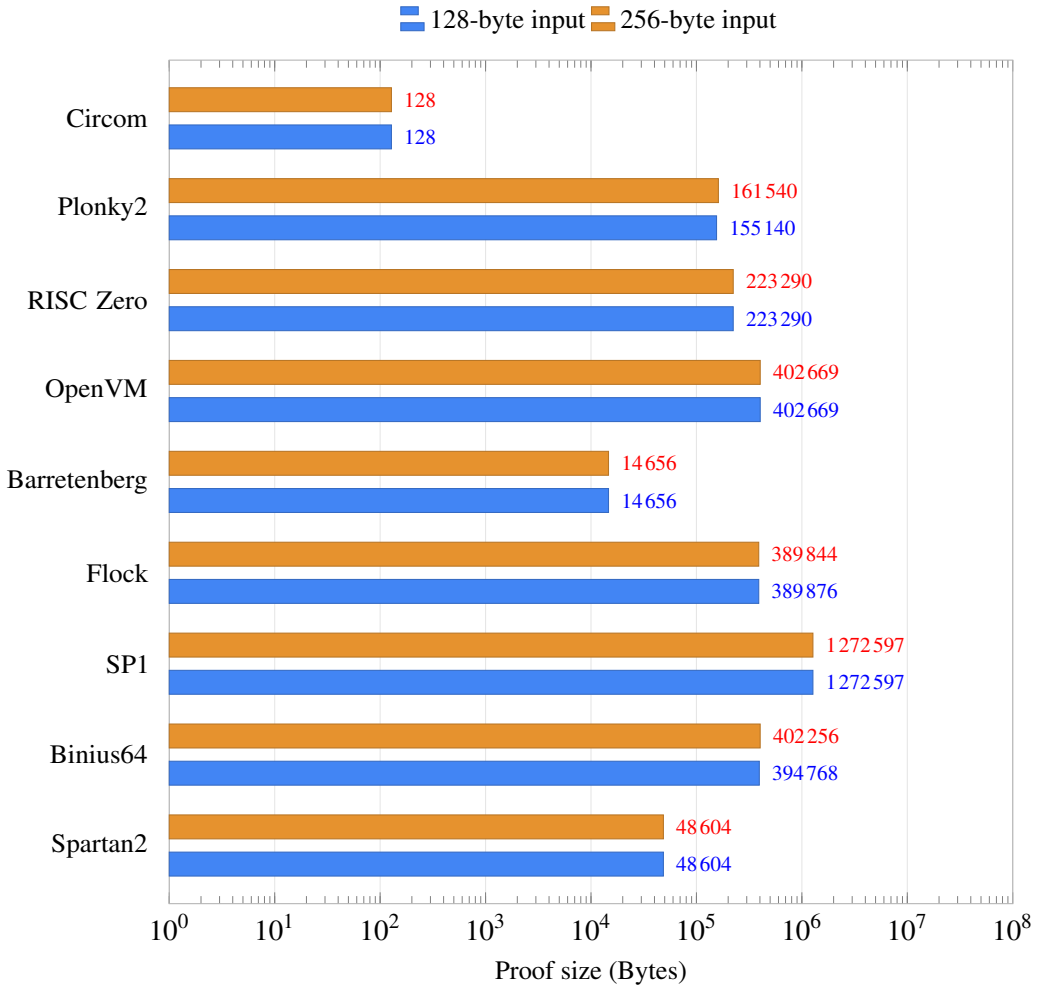


Figure 11: Proof sizes for the SHA256 program

4.3 The streaming solution

The previous benchmarks show that most proofs cannot be handled by a device with tight RAM constraints. A solution to this problem, close to the idea of Section 3.4.3, is to stream the proof. Even if the proof cannot be loaded all at once on the device, some chunks of that proof are independent from the others and can therefore be sent separately. The Verifier device can then proceed with small independent verifications and accept the entire proof if all checks are accepted in sequence.

Let us consider the FRI protocol presented previously: it is composed of the COMMIT and QUERY phases. One can describe the streaming idea at two levels. In a simplified interactive view, the query points are simply values that the Verifier has to remember. In the actual non-interactive proof obtained through the BCS transformation, they are not sent independently: the challenges and query points are derived from the transcript. This distinction is not important for the memory estimate below; in both cases, the Verifier must keep the data that fixes the transcript and the indexed query points, while using only one set of query answers at a time. The query points need not be distinct: a collision may occur when they are derived. Instead, the Verifier must associate every streamed response with its expected query index and position. Since all the t query steps are performed independently and do not need information from the other ones, the Prover does not have to send the t query results all at once. It can be done as follows:

1. The Prover sends the $r+1$ oracle commitments, and the Verifier derives and stores the r challenges from the transcript;

2. The Verifier derives and stores the indexed sequence of t query points from the transcript;
3. For each point:
    - The Prover sends the $3r$ answers to queries and, in a concrete non-interactive proof, their Merkle authentication data;
    - The Verifier checks that the answers correspond to the expected query point, verifies the expected equalities and authentication data, and then deletes the answers.

In the same lower-bound model as above, we would then only have to load at the same time:

$$704 + 1200 + 864 \text{ B} = 2768 \text{ B}$$

This protocol is a good example to present the streaming technique, since it is easy to find which parts of the proof are independent. Other protocols might require a very good understanding for such an analysis, and some could even be completely incompatible with it because of their structure.

5 Verifier memory use

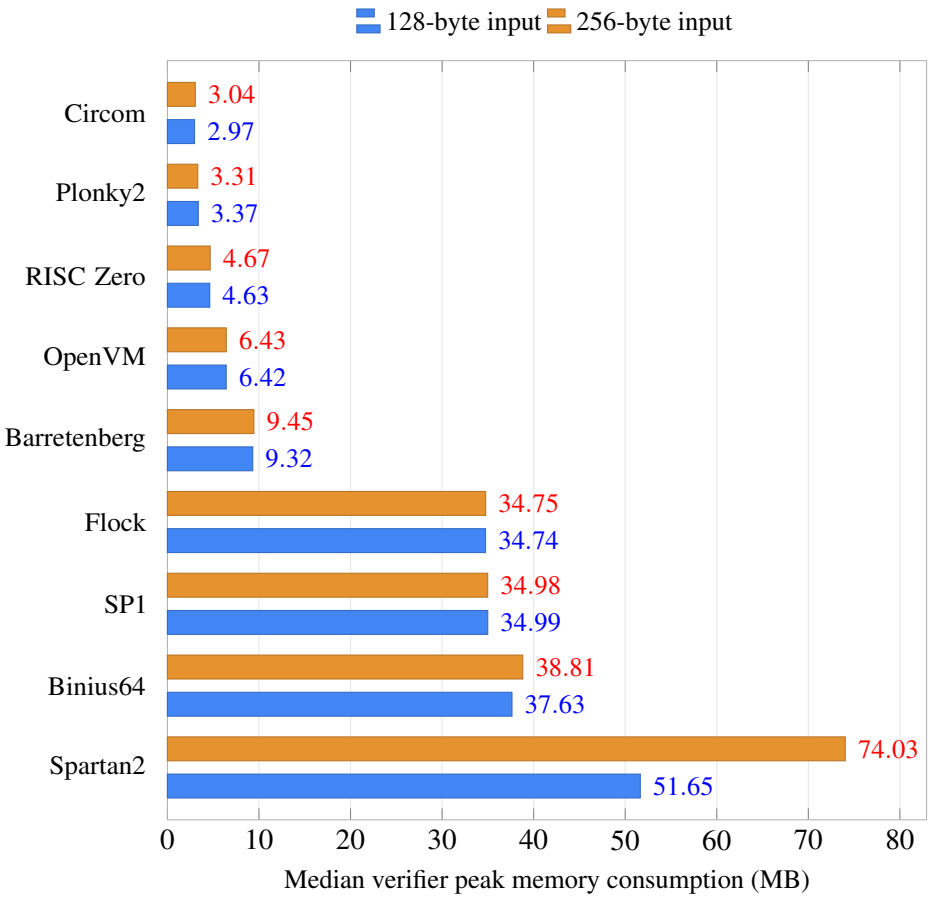


Figure 12: Verifier memory consumption for the SHA256 program

In the previous sections, we studied two families of objects that constrain an embedded Verifier. First, the chosen arithmetization determines which Verifier artifacts must be known in order to authenticate the program. Second, the proof system determines how large the proof is and whether it can be streamed during verification. We now focus on the memory needs of the Verifier during the verification process

itself. This means looking not only at proofs and artifacts loaded directly in RAM, but also at the internal machinery required by the Verifier program for its computations.

All memory values reported in this section are obtained with a verifier-only benchmark. For each proof system and input size, proof generation, setup, witness computation, and verifier artifact preparation are performed before the measurement. The benchmark then runs the optimized verifier as a fresh standalone process, with one warm-up execution followed by several measured executions. We report in Figure 12 the peak memory consumption observed by the operating system for the verifier process only, excluding the benchmarking script and timing supervisor. These numbers therefore measure the practical memory footprint of the implementation on our machine, including loaded artifacts.

The previous sections also showed that some techniques can trade Prover efficiency or proof size for smaller Verifier artifacts, and that proof streaming can reduce the amount of proof data kept in memory at any given time. With that in mind, a first approximation is to subtract the serialized proof and artifact sizes from the measured memory use, as if these objects did not have to be loaded by the Verifier program. This is an optimistic approximation, because serialized sizes do not necessarily match in-memory representations and because memory use also depends on allocator and implementation details. Still, Figure 13 gives a useful lower estimate of the memory cost of the Verifier's computations alone:

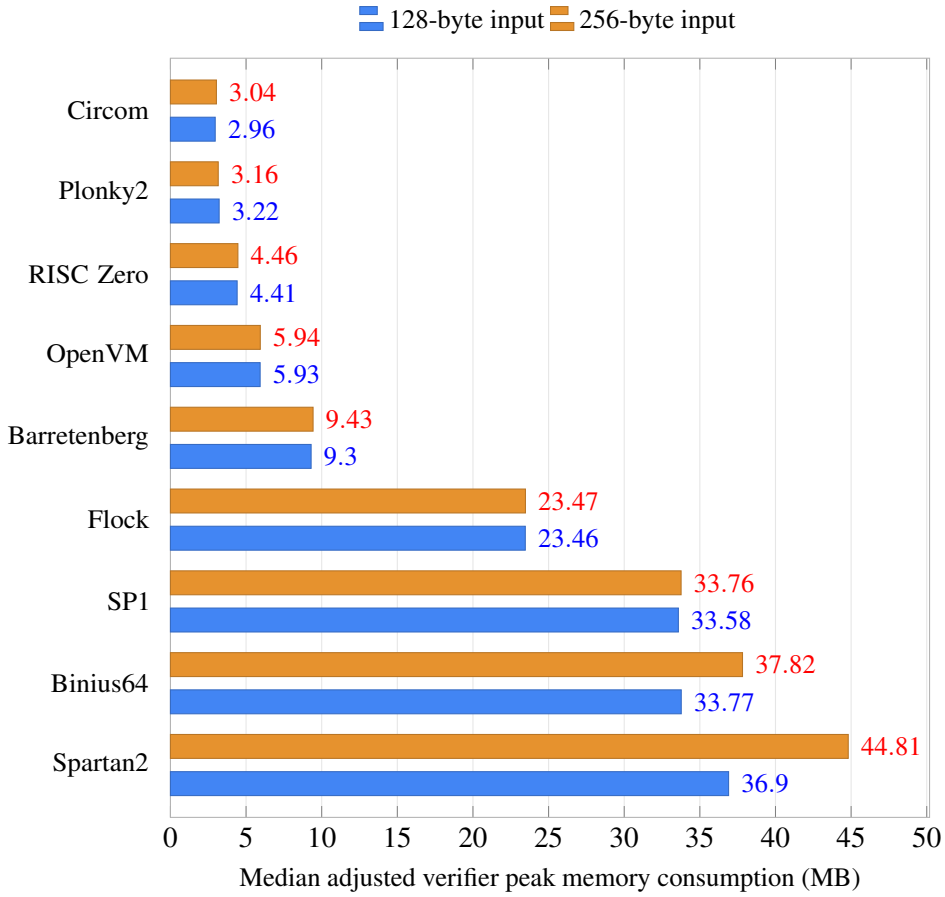


Figure 13: Adjusted verifier memory consumption (computations only) for the SHA256 program

At first sight, the figure may suggest that none of the considered proof systems is light enough to run on a Secure Element once proofs and artifacts are removed from the count. This is not the right conclusion: the figure is mainly a diagnostic tool for estimating how much engineering work may be required to optimize an existing Verifier for a constrained device. For example, ZKNox published ZKlarity in March 2026, an implementation of a Groth16 Verifier on a Ledger Nano with less than 4 kB of usable RAM. Its main difficulty was the implementation of pairing operations under Secure

Element constraints. This example shows that a large desktop memory consumption measurement does not by itself rule out an embedded implementation, because existing libraries are usually designed for conventional operating systems where a few megabytes of memory are not a critical constraint.

However, Groth16 is not post-quantum, whereas post-quantum security is one of the motivations of this work. Among the systems measured above, Plonky2 is therefore the most natural candidate for a proof of concept: its Verifier artifacts are small enough to be embedded for a fixed program, and its proof is too large to fit in RAM at once but has a FRI-based structure that can be streamed. The next section presents this implementation on a Ledger Flex.

6 A PLONKY2 PROOF OF CONCEPT

The previous sections make it possible to estimate the feasibility of an embedded Verifier on a Ledger device. The ZKNox example discussed above shows that such a prototype is possible. Nevertheless, it uses the Groth16 proof system and is therefore not post-quantum, which is an important criterion in our work. Thus, our results led naturally to Plonky2[10], which has small Verifier artifacts and a manageable proof size compared with most post-quantum alternatives considered in the benchmarks.

As a first approach, we consider a simple, visual program called display. From a 4-byte seed, the program generates 32 words of 32 bits using only shift and xor operations. After bitwise decomposition, these words can be interpreted and displayed as a 32×32 bitmap:

Algorithm 2 display program

Require: seed $s \in \{1, \ldots, 2^{32} - 1\}$, image side $N = 32$
Ensure: array of rows $(r_0, \ldots, r_{N-1})$, each $r_i \in \{0,1\}^{32}$

$x \leftarrow s$
for $i = 0$ **to** $N - 1$ **do**
  $x \leftarrow x \oplus (x \ll 13)$
  $x \leftarrow x \oplus (x \gg 17)$
  $x \leftarrow x \oplus (x \ll 5)$
  record the output row $r_i \leftarrow x$ ▷ bit j of r_i is pixel (i, j)
end for
return $(r_0, \ldots, r_{N-1})$

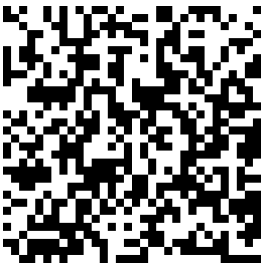


Figure 14: Bitmap output from display for seed 0x12345678

This circuit is not meant to be a realistic privacy application: all the operations are invertible, so the seed can be recovered from the public outputs. Its purpose is rather to obtain a proof whose public result can be checked visually on the device, while still forcing the Verifier to execute the same Plonky2 verification path as for a more realistic circuit.

[10]The upstream implementation is deprecated and no longer receives updates or support. Although it was audited before `v1.0.0`, the implementation presented here should still be understood only as a PoC.

6.1 Streaming the proof and Ledger Flex application

As discussed above, our Secure Element only has a few dozen kB available whereas the Plonky2 proof is on the order of 300 kB. However, a Plonky2 proof has the same two-part shape as the FRI streaming scheme of Section 4.3. In this non-interactive PLONK+FRI setting, the small prefix is the Fiat-Shamir transcript data: commitments, openings, public inputs, and the remaining objects from which the FRI challenges and query positions are derived. The large remainder consists of independent FRI query answers together with their Merkle authentication paths. As in the earlier scheme, the Verifier keeps the prefix, and therefore the derived query positions, in memory; each query can then be checked separately and discarded.

The PoC therefore transmits the proof in this order. The Verifier first reads the transcript prefix and reconstructs the same Fiat-Shamir challenges as the standard Plonky2 Verifier, then receives the FRI queries one after another. This does not modify the mathematical content of the proof: the field elements, commitments, transcript, and checks are the same. Thus, the Verifier performs the same PLONK and FRI verification checks as Plonky2, but organizes them so that the part of the proof growing with the number of queries is never fully stored: only the order in which the Verifier receives the bytes is changed.

The embedded part of the PoC is a Ledger Flex application. In this model, the verifier keys are stored in the device's flash memory and are treated as part of the application. The trust assumption is therefore that these keys are embedded in the authenticated application and are not supplied by the Prover during verification. This corresponds to a fixed set of programs that the device agrees to verify. In the present implementation, two examples are included:

- the display key, whose public inputs are the 32 bitmap rows;
- a sha256 key, whose public input is a SHA-256 digest.

Only one of these keys is used during a verification session. Thus, supporting several examples in the same application does not mean that all verifier data must be simultaneously loaded in RAM.

The proof stream is sent to the Secure Element. The claimed result is supplied by the Prover as part of the public inputs. The embedded Verifier checks the proof with respect to these public inputs; only after the proof is accepted does the application render them as the displayed bitmap or digest.

Figure 15: Ledger Flex PoC screens: app home screen, verified display output, and verified sha256 digest

6.2 Results and limitations

The final application satisfies the Ledger Flex constraints. The application occupies about 220 kB of flash, including the two verifier keys. The streamed proof is about 300 kB, but it is never stored as a whole.

Measurements on a physical Ledger Flex give a verification time of about 36 seconds for both examples. The heap budget is 24 kB, with measured usage slightly below 19 kB, and the peak stack is about 4.2 kB.

The rough estimate of Section 4.3 should therefore not be read as the expected memory cost of this Plonky2 proof. It was a generic STARK/FRI lower-bound model with $r = 12$ folding rounds, $t = 50$ query points, 32-byte hashes, and 24-byte field elements, leading to 2768 B in the streamed setting. In the Plonky2 SHA-256 proof used here, the FRI instance has only 3 commitment rounds, 55 query points, and 7 answer elements per query point in the simplified streamed model. With Goldilocks field elements, the corresponding memory component of the estimate is about 616 B. This reduction comes mainly from the FRI shape: while the simplified estimate used binary folding, Plonky2 uses high-arity reductions, here of arity $2^4 = 16$. Thus three Plonky2 commitment rounds achieve the same degree reduction as twelve binary folding rounds. Together with Plonky2's concrete query and the grinding technique (see [Sta23]), this leads to a smaller streamed FRI component.

When estimating security using the Johnson-bound analysis of Appendix A, the query term must use the same proximity regime as the commitment term. With rate $\rho = 2^{-3}$ and δ close to, but strictly below, $1 - \sqrt{\rho}$, each query contributes at most

$$-\log_2(1-\delta) \simeq -\log_2(\sqrt{\rho}) = \frac{3}{2}$$

bits. Thus a 97-bit target with Plonky2's 16-bit grinding step would require $t = 55$ query points, since

$$55 \times \frac{3}{2} + 16 = 98.5.$$

The commitment soundness term is not the limiting factor in this recalibration: applying the estimate of Appendix A with $r = 3$, the same rate $\rho = 2^{-3}$, and challenges in the quadratic Goldilocks extension, so that $|\mathbb{F}| \simeq 2^{128}$, gives a commitment error around 2^{-98} for compatible choices of δ near the Johnson radius. This should still be read as a consistency check for these parameters rather than as a full independent proof of Plonky2 soundness, since the exact protocol also involves its concrete FRI variant, Fiat-Shamir transformation, Merkle commitments, and implementation-level security accounting.

Remark. A common Plonky2 and ethSTARK security accounting instead assigns $\log_2(1/\rho)$ bits to each query [Sta23]. With $\rho = 2^{-3}$, this gives $28 \times 3 = 84$ query bits, and the 16-bit grinding step then gives the familiar 97-bit estimate. This older estimate comes from a rejected conjecture of FRI soundness up to capacity, i.e., $\delta < 1 - \rho$.

The rejection behavior was also tested. Modified proofs, truncated proofs, proofs with additional data, and proofs associated with the wrong circuit are rejected. This is important because the streaming procedure should not introduce a new acceptance condition: it should only change the memory profile of the usual verification.

This PoC therefore gives a positive answer to the narrow feasibility question: a post-quantum Plonky2 Verifier can be made to run on a Ledger Flex for small fixed circuits, provided that the proof is streamed and the relevant verifier key is embedded in flash. However, it should not be interpreted as a production-ready solution. The verified circuits are fixed at build time, and the verification time remains noticeable for a user-facing device. The contribution is rather to instantiate a plausible model for constrained Verifiers: embed the trusted fixed verifier data in advance, first read the part of the proof that fixes the verifier challenges and query positions, and then process the FRI queries sequentially. Another proof system would require the same idea to be adapted to its own verification structure.

7 Conclusion

The first goal of this work was to evaluate whether building an embedded Verifier program was possible. This led us to work on several proving systems in order to estimate whether the constraints considered

were too strong. The main conclusion of this study is that the most of existing post-quantum proof systems cannot be used directly to embed their Verifier. Indeed, most existing systems are built with Prover performance as the main objective. This is understandable, since verification computations are often negligible compared with proving computations and can be handled by most of today's computers. Unfortunately, this constitutes a major obstacle for our project, since the resources of our Secure Element are not even close to those of an old small computer.

However, we presented here some solutions to deal with large proofs and Verifier artifacts: most of them come with a time trade-off because of a streaming step and/or additional proving steps. The Plonky2 implementation provided a suitable test case for this idea: it uses preprocessed polynomials, and its proof size and structure were compatible with streaming. This led to a working prototype of a Verifier embedded on a Ledger Flex for two lightweight programs.

Even though we succeeded in implementing a post-quantum Verifier on a Secure Element, this remains insufficient at this stage. In fact, Plonky2 is an old proving system that is no longer maintained. Moreover, proving times are too high for practical use when programs become more complex. Potential future directions include the following:

- OpenVM may offer reasonable proof sizes, and its universal design[11] may make the Verifier artifacts negligible. In addition, this project is maintained, and Axiom has announced OpenVM as production-ready;
- Applying the preprocessing technique to some existing proving systems could also be a solution. For systems which are very fast at proof generation, the trade-off might be worth it by reducing the Verifier artifacts. However, the proof size may be affected and become too large for streaming as it's shown in Appendix B;
- Proof systems based on lattice security should also be considered. Very few such proof systems are implemented for now, but they may offer small Verifier artifacts, proof sizes close to the previous ones, and post-quantum security. However, the proving and verifying steps may also take longer.

[11] which is not the one studied here

REFERENCES

[ACFY24] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. WHIR: Reed–solomon proximity testing with super-fast verification. Cryptology ePrint Archive, Paper 2024/1586, 2024.

[BCCT11] Nir Bitansky, Ran Canetti, Alessandro Chiesa, and Eran Tromer. From extractable collision resistance to succinct non-interactive arguments of knowledge, and back again. Cryptology ePrint Archive, Paper 2011/443, 2011.

[BGK+23] Alexander R. Block, Albert Garreta, Jonathan Katz, Justin Thaler, Pratyush Ranjan Tiwari, and Michał Zając. Fiat-shamir security of FRI and related SNARKs. Cryptology ePrint Archive, Paper 2023/1071, 2023.

[BRW26] Benedikt Bünz, Ron Rothblum, and William Wang. Flock: Fast proving for batch boolean computations. Cryptology ePrint Archive, Paper 2026/1329, 2026.

[BSBHR18a] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast Reed-Solomon Interactive Oracle Proofs of Proximity, 2018.

[BSBHR18b] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Paper 2018/046, 2018.

[BSCH+25] Eli Ben-Sasson, Dan Carmon, Ulrich Haböck, Swastik Kopparty, and Shubhangi Saraf. On proximity gaps for reed–solomon codes. Cryptology ePrint Archive, Paper 2025/2055, 2025.

[BSCI+20] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed-solomon codes. Cryptology ePrint Archive, Paper 2020/654, 2020.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. Cryptology ePrint Archive, Paper 2016/116, 2016.

[BSGKS19] Eli Ben-Sasson, Lior Goldberg, Swastik Kopparty, and Shubhangi Saraf. DEEP-FRI: Sampling outside the box improves soundness. Cryptology ePrint Archive, Paper 2019/336, 2019.

[CY24] Alessandro Chiesa and Eylon Yogev. *Building Cryptographic Proofs from Hash Functions*. 2024.

[DP24] Benjamin E. Diamond and Jim Posen. Polylogarithmic proofs for multilinears over binary towers. Cryptology ePrint Archive, Paper 2024/504, 2024.

[Fu25] Shihui Fu. Improved constant-sized polynomial commitment schemes without trusted setup. Cryptology ePrint Archive, Paper 2025/1233, 2025.

[GMW25] Albert Garreta, Nicolas Mohnblatt, and Benedikt Wagner. A simplified round-by-round soundness proof of FRI. Cryptology ePrint Archive, Paper 2025/1993, 2025.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Proceedings, Part II, of the 35th Annual International Conference on Advances in Cryptology — EUROCRYPT 2016 - Volume 9666*, page 305–326. Springer-Verlag, 2016.

[Gur04] Venkatesan Guruswami. *List Decoding of Error-Correcting Codes: Winning Thesis of the 2002 ACM Doctoral Dissertation Competition*, volume 3282 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2004.

[Gur07] Venkatesan Guruswami. *Algorithmic results in list decoding*, volume 2(2) of *Foundations and Trends® in Theoretical Computer Science*. Now Publishers Inc., 2007.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Paper 2019/953, 2019.

[Hab22] Ulrich Haböck. A summary on the FRI low degree test. Cryptology ePrint Archive, Paper 2022/1216, 2022.

[HLP24] Ulrich Haböck, David Levit, and Shahar Papini. Circle STARKs. Cryptology ePrint Archive, Paper 2024/278, 2024.

[KPV19] Assimakis Kattis, Konstantin Panarin, and Alexander Vlasov. RedShift: Transparent SNARKs from list polynomial commitments. Cryptology ePrint Archive, Paper 2019/1400, 2019.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194. Springer Berlin Heidelberg, 2010.

[LCH14] Sian-Jheng Lin, Wei-Ho Chung, and Yunghsiang S. Han. Novel polynomial basis and its application to reed-solomon erasure codes. In *2014 IEEE 55th Annual Symposium on Foundations of Computer Science*, pages 316–325, 2014.

[Mar26] Quentin Mareau. Construction of a snark from the fri protocol. https://mareauq.github.io/_pages/perso/papiers/FRI-SNARK_report.pdf, 2026.

[MF23] Tiago Martins and João Farinha. Study of arithmetization methods for STARKs. Cryptology ePrint Archive, Paper 2023/661, 2023.

[SB25a] Whiteboard Sessions and Dan Boneh. Zk whiteboard sessions - S2M7: FRI and proximity proofs (part 1), 2025.

[SB25b] Whiteboard Sessions and Dan Boneh. Zk whiteboard sessions - S2M7: FRI and proximity proofs (part 2), 2025.

[SG23] Colin Steidtmann and Sanjay Gollapudi. Benchmarking ZK-circuits in circom. Cryptology ePrint Archive, Paper 2023/681, 2023.

[Sta23] StarkWare. ethSTARK documentation – version 1.2. Cryptology ePrint Archive, Paper 2021/582, 2023.

[STW23] Srinath Setty, Justin Thaler, and Riad Wahby. Customizable constraint systems for succinct arguments. Cryptology ePrint Archive, Paper 2023/552, 2023.

[ZCF23] Hadas Zeilberger, Binyi Chen, and Ben Fisch. BaseFold: Efficient field-agnostic polynomial commitment schemes from foldable codes. Cryptology ePrint Archive, Paper 2023/1705, 2023.

A FURTHER DETAILS ON FRI

This appendix gives the technical details behind the simplified FRI presentation used in Section 4.3. It describes the folding procedure, the interactive protocol, its soundness bound, and a FRI-based PCS. A more precise description of each of these points can be found in the [Mar26].

A.1 Folding

An important technique in the construction of FRI is called *folding*. In general, a folding of order m consists of decomposing a polynomial $P \in \mathbb{F}^{<dm}[X]$ into m polynomials $P_1, \dots, P_m \in \mathbb{F}^{<d}[X]$, and then studying the polynomial

$$\mathsf{Fold}(P, \alpha) = P_1 + \alpha P_2 + \cdots + \alpha^{m-1} P_m.$$

Such a construction has useful properties with respect to Reed-Solomon codes and allows the degree of the polynomials to decrease geometrically. More general constructions, together with their properties, are presented in [ZCF23], [BSGKS19], and [GMW25]. To give a simple presentation of the FRI protocol, we restrict ourselves here to the study of the classical folding of order 2. When there is no ambiguity, we simply speak of folding.

Let us begin by briefly describing what our classical folding consists of. For an even integer d and $f \in \mathbb{F}^{<d}[X]$, we consider the polynomials $f_{\text{even}}, f_{\text{odd}} \in \mathbb{F}^{<d/2}[X]$, constructed respectively from the even-degree and odd-degree monomials of f.

Example. For $f = 2X^5 + 3X^4 - X^2 + 5X - 3$, we get

$$f_{\text{even}} = 3X^2 - X - 3 \qquad \text{and} \qquad f_{\text{odd}} = 2X^2 + 5.$$

This decomposition satisfies, in particular,

$$f(X) = f_{\text{even}}(X^2) + X f_{\text{odd}}(X^2),$$

and also

$$f_{\text{even}}(X^2) = \frac{f(X) + f(-X)}{2} \qquad \text{and} \qquad f_{\text{odd}}(X^2) = \frac{f(X) - f(-X)}{2X}.$$

The key points are the following:

- For every $z \in \mathbb{F}^*$, one can evaluate $f_{\text{even}}(z^2)$ and $f_{\text{odd}}(z^2)$ using only $f(z)$ and $f(-z)$.
- This construction can be carried out for any oracle, and not only for polynomials. More precisely, for an oracle $u \in \mathbb{F}^{\mathcal{L}}$, an element $r \in \mathbb{F}$, and a point $a \in \mathcal{L}$, we define

$$\mathsf{Fold}(u, r)(a^2) := \frac{u(a) + u(-a)}{2} + r \frac{u(a) - u(-a)}{2a}.$$

The study of the new oracle obtained in this way is the starting point of the construction of FRI.

For our construction of FRI, it is necessary to consider a domain $\mathcal{L}_0$ that is a multiplicative coset in $\mathbb{F}^*$. That is, $\mathcal{L}_0 = aH$ for $a \in \mathbb{F}^*$ and H a multiplicative subgroup of $\mathbb{F}^*$. We must also ensure that $|\mathcal{L}_0| = n = 2^k$ for $k \in \mathbb{N}^*$. Since H has even order, we have $\text{char}(\mathbb{F}) \neq 2$ and $-1 \in H$. Hence, for every $a \in \mathcal{L}_0$, we also have $-a \in \mathcal{L}_0$, and the divisions by 2 in the folding formulas are well-defined. It is worth noting that all these conditions affect the choice of the field $\mathbb{F}$. We also define

$$\forall i \in \{1, \dots, k\}, \qquad \mathcal{L}_i = \{x^{2^i},\ x \in \mathcal{L}_0\}.$$

For every $i < k$, the domain $\mathcal{L}_i$ is likewise closed under negation, so the same property holds in each folding round.

A.2 Protocol presentation

The protocol is presented in oracle form: the Prover's messages are treated as oracles that the Verifier accesses only through queries. In practice, each oracle is represented by a short commitment, which allows the Verifier to check queried values without reading the whole oracle. For an oracle f, we denote such a commitment by $[f]$. Further details can be found in the [Mar26]. We fix a domain $\mathcal{L}_0$ as a coset of a multiplicative subgroup of $\mathbb{F}^*$ of order $n = 2^k$, an oracle $f_0 : \mathcal{L}_0 \longrightarrow \mathbb{F}$, and a rate $\rho = 2^{-\mathcal{R}}$, where $1 \leq \mathcal{R} < k$. The FRI protocol then consists of two phases, COMMIT and QUERY, whose purpose is to convince the Verifier that f_0 is close to $\mathcal{C}_0 = \mathrm{RS}[\mathbb{F}, \mathcal{L}_0, d]$, with $d = \rho|\mathcal{L}_0| = 2^{k-\mathcal{R}}$.
We now fix an integer r such that $1 \leq r \leq k - \mathcal{R}$. The COMMIT phase then has r steps and allows the Prover to provide the Verifier with a sequence of oracle commitments. For every $i \in \{1, \ldots, r\}$, we set $\mathcal{C}_i = \mathrm{RS}[\mathbb{F}, \mathcal{L}_i, d/2^i]$.
The QUERY phase uses $t \geq 1$ query points to convince the Verifier that the oracles committed by the Prover have indeed been obtained by successive foldings and that the final oracle is a word of the code $\mathcal{C}_r$. In practice, this last verification remains easy provided that r is chosen so that the final domain $\mathcal{L}_r$ has small cardinality.
$\mathcal{L}_0$, ρ, and t are FRI parameters, and their choices affect the security of the protocol.

We now describe the two phases of the FRI protocol:

Phase COMMIT:

1. The Prover sends the commitment $[f_0]$.
2. For every round $i = 1, \ldots, r$:
   (i) The Verifier sends a challenge $\alpha_i \in \mathbb{F}$.
   (ii) The Prover returns the commitment $[f_i]$ with $f_i = \mathsf{Fold}(f_{i-1}, \alpha_i)$.

Phase QUERY:

3. The Verifier chooses $s_1, \ldots, s_t \in \mathcal{L}_0$ and sends $\alpha_{r+1} = (s_1, \ldots, s_t)$ to the Prover.
4. The Verifier reads all values of f_r on $\mathcal{L}_r$ and checks that $f_r \in \mathcal{C}_r$.
5. For every $j = 1, \ldots, t$:
   - $s \leftarrow s_j$.
   - For every $i = 1, \ldots, r$:
     (i) The Verifier queries f_{i-1} at the values $s^{2^{i-1}}$ and $-s^{2^{i-1}}$, then computes

$$\mathsf{Fold}(f_{i-1}, \alpha_i)(s^{2^i}) = \frac{f_{i-1}(s^{2^{i-1}}) + f_{i-1}(-s^{2^{i-1}})}{2} + \alpha_i \frac{f_{i-1}(s^{2^{i-1}}) - f_{i-1}(-s^{2^{i-1}})}{2s^{2^{i-1}}}.$$

     (ii) The Verifier queries f_i at the value s^{2^i} and checks that

$$\mathsf{Fold}(f_{i-1}, \alpha_i)(s^{2^i}) = f_i(s^{2^i}).$$

PROTOCOL 1 - FRI

A.3 Soundness bound and security parameters

The soundness analysis of FRI is based on the correlated agreement property and on results from [BSCH+25] and [GMW25]. For a Reed-Solomon code of rate ρ and a proximity parameter $\delta \in]0, 1-\sqrt{\rho}[$, we set

$$\eta = \max\left(\left\lceil \frac{\sqrt{\rho}}{1-\sqrt{\rho}-\delta} \right\rceil, 3\right) + \frac{1}{2}.$$

Proposition A.1 (Soundness of FRI under the Johnson bound)**.** *For the construction of the FRI protocol with r COMMIT rounds and t QUERY points, and for $0 < \delta < 1-\sqrt{\rho}$, we have*

$$\varepsilon_{FRI} \leq \varepsilon_C + \varepsilon_Q$$

where

$$\varepsilon_C \leq \frac{1}{|\mathbb{F}|}\left((2^r-1)\frac{2\eta^5+3\eta\delta\rho}{3\rho^{5/2}} + \frac{r\eta}{\sqrt{\rho}}\right)$$

and

$$\varepsilon_Q < (1-\delta)^t.$$

Thus, for a security parameter λ, it is usual to check that

$$\varepsilon_C \leq \frac{2^{-\lambda}}{2} \qquad \text{and} \qquad \varepsilon_Q \leq \frac{2^{-\lambda}}{2}$$

thereby guaranteeing that $\varepsilon_{FRI} = \varepsilon_C + \varepsilon_Q \leq 2^{-\lambda}$. Moreover, a larger δ implies a smaller t and therefore fewer Verifier queries, which ultimately leads to a smaller proof after the BCS compilation. This is why it is necessary to study the best compromise between ε_C and ε_Q.

A.4 FRI-based Polynomial Commitment Scheme

We now instantiate the IOPP of the generic PCS construction with FRI. Throughout this construction, we use an oracle-commitment model: a commitment C fixes an oracle, and the Verifier can later query values of this oracle consistently with C. More precisely, when a commitment C is fixed, we denote by $u_C : \mathcal{L} \to \mathbb{F}$ the oracle associated with it.
The quotient construction presented below is an instance of the *Domain Extension for Eliminating Pretenders* technique, abbreviated as **DEEP**. It transforms evaluation claims at points outside $\mathcal{L}$ into a new Reed-Solomon proximity claim on $\mathcal{L}$.

Let $\mathcal{L} \subseteq \mathbb{F}$, let $f : \mathcal{L} \longrightarrow \mathbb{F}$ be an oracle, let $\Omega = (z_1, \ldots, z_m)$ be an m-tuple of pairwise distinct points in $\mathbb{F} \setminus \mathcal{L}$, and let $\mathbf{y} = (y_1, \ldots, y_m) \in \mathbb{F}^m$. We then define

$$q = \mathrm{Quot}(f, \Omega, \mathbf{y}) : \mathcal{L} \longrightarrow \mathbb{F}$$

by

$$\forall x \in \mathcal{L}, \qquad q(x) = \frac{f(x) - I_\Omega(x)}{Z_\Omega(x)}$$

where

$$Z_\Omega(X) = \prod_{i=1}^{m}(X - z_i) \in \mathbb{F}[X]$$

and I_Ω denotes the unique polynomial of degree strictly less than m satisfying

$$\forall i \in \{1, \ldots, m\}, \qquad I_\Omega(z_i) = y_i.$$

Lemma A.2. *Let $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$, $\delta \in]0,1[$, $u \in \mathbb{F}^{\mathcal{L}}$, an integer $m < d$, an m-tuple $\Omega = (z_1, \ldots, z_m)$ of pairwise distinct points in $\mathbb{F} \setminus \mathcal{L}$ and $\mathbf{y} = (y_1, \ldots, y_m) \in \mathbb{F}^m$. For $q = \mathrm{Quot}(u, \Omega, \mathbf{y})$, the following assertions are equivalent:*

1. *There exists $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q, Q|_{\mathcal{L}}) \leq \delta$.*
2. *There exists $P \in \mathbb{F}^{<d}[X]$ such that $\Delta(u, P|_{\mathcal{L}}) \leq \delta$ and $P(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$.*

Proof. If such a $Q \in \mathbb{F}^{<d-m}[X]$ exists, we set

$$P(X) = Q(X)Z_\Omega(X) + I_\Omega(X).$$

Since $Z_\Omega(x) \neq 0$ for every $x \in \mathcal{L}$, multiplying pointwise by Z_Ω and adding I_Ω preserves the positions of disagreements. The two resulting words are respectively u and $P|_{\mathcal{L}}$, hence

$$\Delta(q, Q|_{\mathcal{L}}) = \Delta(u, P|_{\mathcal{L}}).$$

Moreover, we indeed have $\deg P < d$ and, for every $i \in \{1, \ldots, m\}$,

$$P(z_i) = Q(z_i)Z_\Omega(z_i) + I_\Omega(z_i) = y_i.$$

Conversely, if such a $P \in \mathbb{F}^{<d}[X]$ exists, then the polynomial $P - I_\Omega$ vanishes at each of the z_i, hence Z_Ω divides $P - I_\Omega$. Thus,

$$Q = \frac{P - I_\Omega}{Z_\Omega}$$

is indeed a polynomial of degree strictly less than $d - m$. The same computations as those made above then give $\Delta(q, Q|_{\mathcal{L}}) \leq \delta$. □

The interpretation of this lemma is the following: checking that u is δ-close to the restriction of a polynomial P of degree $< d$ satisfying $P(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$ simply amounts to checking that $\mathrm{Quot}(u, \Omega, \mathbf{y})$ is δ-close to $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d - m]$. In particular, we have

Corollary A.3. *Let $m < d$, $u \in \mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$ and $P \in \mathbb{F}^{<d}[X]$ be the polynomial that extends u over $\mathbb{F}$. If $P(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$, then $\mathrm{Quot}(u, \Omega, \mathbf{y}) \in \mathrm{RS}[\mathbb{F}, \mathcal{L}, d - m]$.*

First, computing $q(x)$ for $x \in \mathcal{L}$ and $q = \mathrm{Quot}(u, \Omega, \mathbf{y})$ requires no additional query compared with querying the Prover directly for $u(x)$. Thus, if an interactive protocol convinces the Verifier that q is δ-close to $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d - m]$, then, up to the protocol soundness error, the Verifier learns that there exists a polynomial $P \in \mathbb{F}^{<d}[X]$ whose restriction to $\mathcal{L}$ is δ-close to u and which satisfies $P(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$. We therefore obtain an additional property with essentially the same amount of computation.

These properties give the main idea behind the FRI-based PCS: using FRI to perform a proximity test on a quotient oracle allows the Verifier to be convinced, up to soundness error, that there exists a polynomial close to u_C that satisfies the expected properties. To obtain a classical PCS, this polynomial must also be unique; we therefore define the PCS only in the *unique decoding regime*, where $\delta \leq (1 - \rho)/2$.

Let us now present the protocol:

- $\mathsf{pp} \leftarrow \mathsf{Setup}(1^\lambda, d, m_{\max})$: the public parameters pp fix a domain $\mathcal{L}$ and a proximity radius δ. They must satisfy

$$0 < \delta \leq \frac{1-\rho}{2} \qquad \text{and} \qquad m_{\max} < d \qquad \text{with} \qquad 0 < \rho = \frac{d}{|\mathcal{L}|} < 1.$$

- $C \leftarrow \mathsf{Commit}(\mathsf{pp}, d, f(X))$: for $f \in \mathbb{F}^{<d}[X]$, we fix the oracle $u_f \in \mathbb{F}^{\mathcal{L}}$ defined by $u_f(x) = f(x)$ for every $x \in \mathcal{L}$. In the oracle model, the commitment C is an ideal handle to this oracle; in an implementation, it is represented by a short commitment $[u_f]_{x\in\mathcal{L}}$.

- $b \in \{0,1\} \leftarrow \mathsf{Open}(\mathsf{pp}, d, C, f(X))$: returns 1 if the oracle u_C associated with C is at relative distance at most δ from $f|_{\mathcal{L}}$, and 0 otherwise.

The algorithm Eval relies on a FRI instance associated with a Reed-Solomon code. The DEEP quotient construction above, and in particular Lemma A.2, allows it to be described through the quotient oracle associated with the oracle u_C:

- We fix $m \leq m_{\max}$, an m-tuple $\Omega = (z_1, \ldots, z_m)$ of pairwise distinct points in $\mathbb{F} \setminus \mathcal{L}$ and $\mathbf{y} = (y_1, \ldots, y_m)$, and then define the oracle

$$q_{u_C} = \mathrm{Quot}(u_C, \Omega, \mathbf{y}) \in \mathbb{F}^{\mathcal{L}}.$$

  We then set $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$ and run the protocol $\mathrm{FRI}(q_{u_C}, \mathcal{C})$, with rate $(d-m)/|\mathcal{L}|$, which returns accept or reject. We then define

$$b \in \{0,1\} \leftarrow \mathsf{Eval}(\mathsf{pp}, d, C, \Omega, \mathbf{y}; f(X))$$

  by setting $b = 1$ if $\mathrm{FRI}(q_{u_C}, \mathcal{C}) \rightarrow \mathsf{accept}$, and $b = 0$ otherwise.

PROTOCOL 2 - FRI-based PCS

Remark. The quotient oracle q_{u_C} should be understood as a virtual oracle derived from the committed oracle u_C. In the oracle model, whenever FRI queries q_{u_C} at a point $x \in \mathcal{L}$, the Verifier queries $u_C(x)$ and computes

$$q_{u_C}(x) = \frac{u_C(x) - I_\Omega(x)}{Z_\Omega(x)}.$$

After Merkle compilation, this means that the Prover opens $u_C(x)$ with respect to the root representing C, while the folded oracles required by FRI are authenticated by their own Merkle roots. Thus, the Prover is not allowed to choose an independent initial oracle for the proximity test: every queried value of the first FRI oracle is checked against the value computed from the commitment C.

Lemma A.4. *Let ε_{FRI} be the soundness error of the FRI instance used for the code $\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m]$. If no polynomial $f \in \mathbb{F}^{<d}[X]$ whose restriction to $\mathcal{L}$ is at relative distance at most δ from u_C satisfies $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$, then, for every possibly malicious Prover $\tilde{P}$ in the FRI instance with initial oracle q_{u_C},*

$$\mathbb{P}\left[\left\langle \tilde{P}(q_{u_C}, \mathcal{C}),\, V(\mathcal{C}) \right\rangle = \mathsf{accept}\right] \leq \varepsilon_{FRI}.$$

Proof. Suppose that no polynomial $f \in \mathbb{F}^{<d}[X]$ simultaneously satisfies $\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$. If there existed a polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q|_{\mathcal{L}}) \leq \delta$, then Lemma A.2 would provide a polynomial $f \in \mathbb{F}^{<d}[X]$ satisfying $\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$, which would contradict the hypothesis. We therefore have

$$\Delta(q_{u_C}, \mathcal{C}) \geq \delta.$$

The soundness of FRI applied to the initial oracle q_{u_C} and to the code $\mathcal{C}$ then gives the announced bound. □

Finally, the main result is as follows:

Theorem A.5. *Under the oracle-commitment model above, for every choice of $m \le m_{\max}$, if the FRI parameters yield a soundness error $\varepsilon_{FRI} \le 2^{-\lambda}$, then the protocol above defines a Polynomial Commitment Scheme for evaluation queries over $\mathbb{F} \setminus \mathcal{L}$.*

Proof. We verify the properties required of a PCS in this oracle-commitment model.
Completeness. If C is the ideal commitment to u_f with $u_f(x) = f(x)$ for every $x \in \mathcal{L}$, and if $f(z_i) = y_i$ for every $i \in \{1,\dots,m\}$, then $u_C = u_f$ and Corollary A.3 ensures that $q_{u_C} \in \mathcal{C}$. Completeness then follows directly from that of FRI.
Binding. Let $f_1, f_2 \in \mathbb{F}^{<d}[X]$ and let C be a commitment. If

$$\mathsf{Open}(\mathsf{pp}, d, C, f_1(X)) = 1 \qquad \text{and} \qquad \mathsf{Open}(\mathsf{pp}, d, C, f_2(X)) = 1,$$

then, by definition of Open,

$$\Delta(u_C, f_1|_{\mathcal{L}}) \le \delta \qquad \text{and} \qquad \Delta(u_C, f_2|_{\mathcal{L}}) \le \delta.$$

Since $\delta \le \dfrac{1-\rho}{2}$, we are in the unique decoding regime for the code $\mathrm{RS}[\mathbb{F}, \mathcal{L}, d]$, and there is at most one polynomial of degree strictly less than d at relative distance at most δ from u_C. We deduce that $f_1 = f_2$, so the binding property holds with error 0 in this oracle-level model.
Knowledge extractability. Fix an admissible public instance $\mathsf{x} = (\mathsf{pp}, d, C, \Omega, \mathbf{y})$ and a malicious Prover $\tilde{P}$ for the protocol Eval. We denote

$$\rho' = \frac{d-m}{|\mathcal{L}|} \le \rho.$$

Since $\delta \le \dfrac{1-\rho}{2} \le \dfrac{1-\rho'}{2}$, we remain within the unique decoding radius of the code

$$\mathcal{C} = \mathrm{RS}[\mathbb{F}, \mathcal{L}, d-m].$$

We define an oracle-level polynomial-time extractor E which, on input x, reads the oracle u_C associated with the ideal commitment C, then computes

$$q_{u_C} = \mathrm{Quot}(u_C, \Omega, \mathbf{y}) \in \mathbb{F}^{\mathcal{L}},$$

and applies a Reed-Solomon unique decoder, for example the Berlekamp-Welch algorithm, to the word q_{u_C} with respect to $\mathcal{C}$. If decoding fails, the extractor returns $\perp$. Otherwise, it obtains the unique polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q|_{\mathcal{L}}) \le \delta$, then constructs

$$f(X) = Q(X) Z_\Omega(X) + I_\Omega(X) \in \mathbb{F}^{<d}[X]$$

and returns $f(X)$. The implication $(1) \Rightarrow (2)$ of Lemma A.2 then shows that

$$\Delta(u_C, f|_{\mathcal{L}}) \le \delta \qquad \text{and} \qquad f(z_i) = y_i \ \text{ for every } i \in \{1,\dots,m\}.$$

By definition,

$$\mathsf{Open}(\mathsf{pp}, d, C, f(X)) = 1,$$

hence

$$(\mathsf{x}, f(X)) \in \mathcal{R}_{\mathsf{Eval}}.$$

Now suppose that the extractor returns $\perp$. By correctness of unique decoding, there is then no polynomial $Q \in \mathbb{F}^{<d-m}[X]$ such that $\Delta(q_{u_C}, Q|_{\mathcal{L}}) \le \delta$. The converse implication $(2) \Rightarrow (1)$ of Lemma A.2 therefore

shows that there is no polynomial $f \in \mathbb{F}^{<d}[X]$ simultaneously satisfying $\Delta(u_C, f|_{\mathcal{L}}) \leq \delta$ and $f(z_i) = y_i$ for every $i \in \{1, \ldots, m\}$. Hence there is no witness for the relation $\mathcal{R}_{\mathsf{Eval}}$. Lemma A.4 then implies

$$\mathbb{P}\left[\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\rangle = \mathsf{accept}\right] \leq \varepsilon_{FRI}.$$

Thus, in all cases,

$$\mathbb{P}\left[(\mathsf{x}, E^{\tilde{P}}(\mathsf{x})) \in \mathcal{R}_{\mathsf{Eval}}\right] \geq \mathbb{P}\left[\langle \tilde{P}(\mathsf{x}),\, V(\mathsf{x})\rangle = \mathsf{accept}\right] - \varepsilon_{FRI}.$$

This shows that Eval is an argument of knowledge for $\mathcal{R}_{\mathsf{Eval}}$ with knowledge-soundness error $\varepsilon_{FRI} \leq 2^{-\lambda}$. □

Remark. The construction above handles PCS evaluation claims only at points outside the domain $\mathcal{L}$, where the quotient oracle is well defined. Once the protocol has accepted, soundness guarantees, except with probability at most ε_{FRI}, that the committed oracle u_C is δ-close to a unique polynomial $P \in \mathbb{F}^{<d}[X]$ satisfying the claimed out-of-domain evaluations. In particular, u_C and P agree on at least $(1-\delta)|\mathcal{L}|$ points of $\mathcal{L}$. Thus, after this PCS verification, one may still query the committed oracle at a point $z \in \mathcal{L}$ through the commitment C, but this proves the value of $u_C(z)$ rather than a PCS evaluation claim for $P(z)$ at a fixed or adversarially chosen point.
However, this limitation is not an issue in our context, although the construction is somewhat too sophisticated for our needs. Indeed, for preprocessed artifacts, the Verifier trusts the commitment to represent a polynomial and only needs to certify evaluations at random challenge points in the field. Consequently, the committed oracle is known to be a codeword, so one may consider $(1-\rho)/2 < \delta < 1-\rho$, since all distinct codewords are at relative distance at least $1-\rho$. Under the FRI security constraint, we can work above the unique decoding radius, but must remain below the Johnson bound: $0 < \delta < 1 - \sqrt{\rho}$.

B AN EXAMPLE ON PREPROCESSED BINIUS64

In section 3, we suggested some ways of handling large Verifier artifacts. Amoung them, we suggested replacing them with commitments during preprocessing, or authenticating them while they are streamed. This example applies the first idea to Binius64. The goal here is to measure the trade off between Verifierartifacts size and proof size on one of the systems whose raw artifacts are large.

More precisely, the large verifier artifact is replaced by a compact commitment, and the Prover adds the data needed to open the committed object during verification. The resulting measurements are shown below:

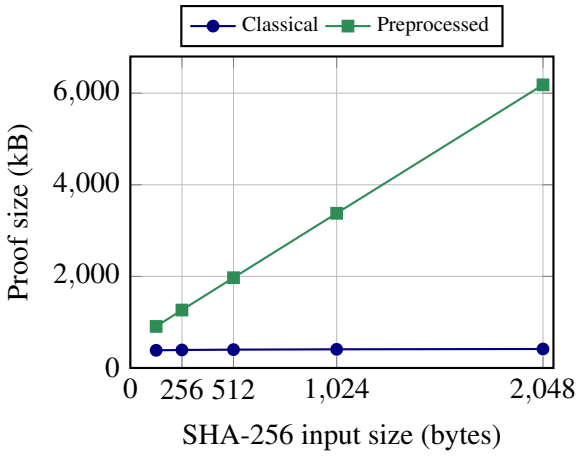


Figure 16: Binius64 proof size, with and without preprocessing

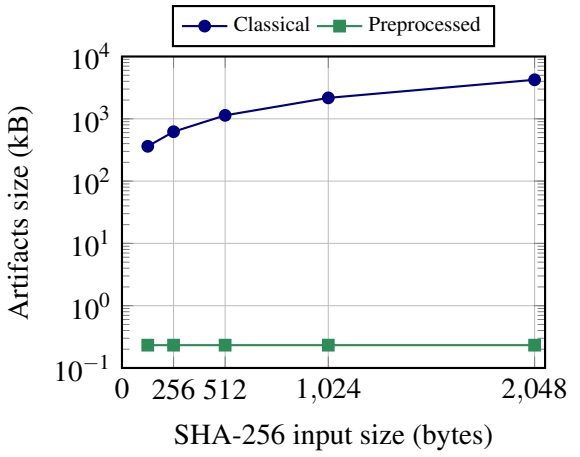


Figure 17: Binius64 verifier artifact size, with and without preprocessing

As expected, preprocessing almost removes the trusted artifact from the Verifier's point of view: in these measurements, it is reduced to 238 bytes, while the original artifact ranges from about 362 kB to 4.13 MB. Even if this represents a huge reduction, it is paid for inside the proof. The classical Binius64 proof stays close to 400 kB over the tested inputs, whereas the preprocessed proof grows almost linearly with the input size: from about 907 kB to 6.04 MB.

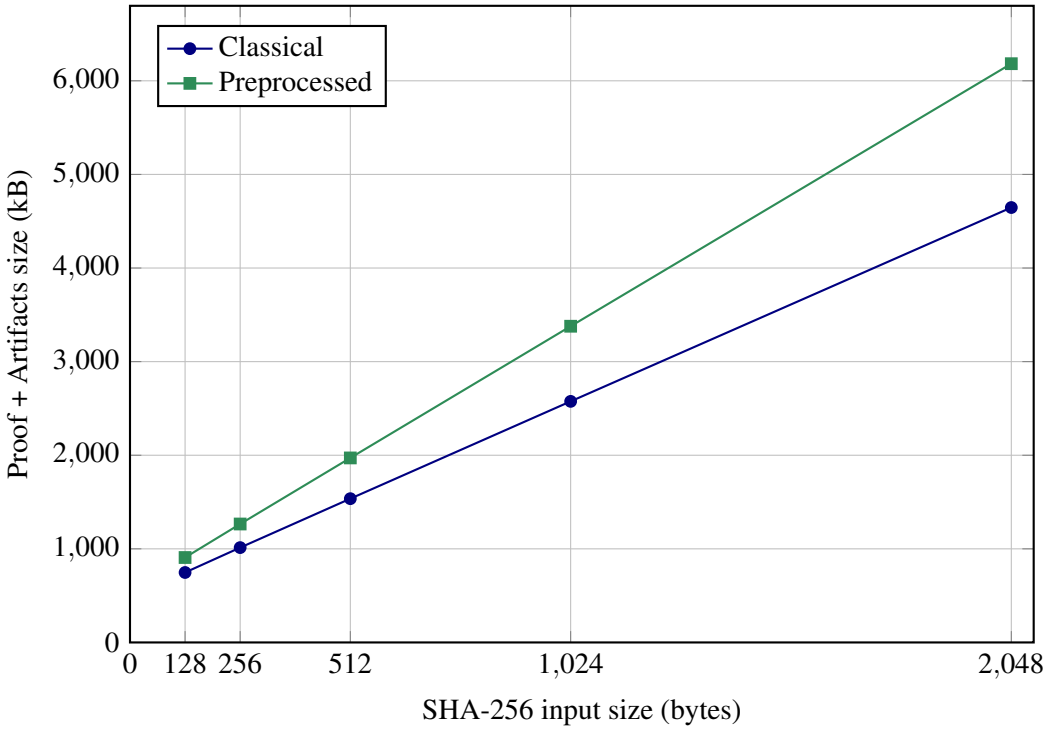


Figure 18: Comparison on Binius64 proof + Verifier artifacts size with and without preprocessing

The last figure gives the relevant comparison when the proof and the Verifier artifacts must both be supplied to the device. In this experiment, preprocessing increases the total amount of data by about 21% to 33%: this means roughly 160 kB of extra data for a 128-byte input and about 1.5 MB for a 2048-byte input. By contrast, the authenticated-streaming construction of Section 3.4.3 would add about 32 kB in a pessimistic case. Therefore, for Binius64, preprocessing does not seem to be the best way to reduce the artifact size. However, it is important to keep in mind that the streaming option can only be used with a detailed understanding of the specific arithmetization: it must be splittable during the verification phase.